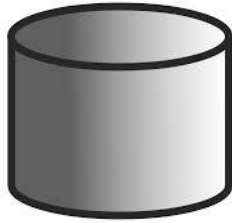


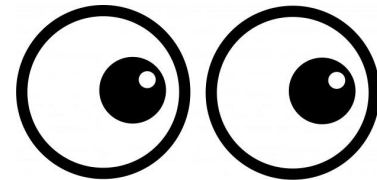
DATABASES



Unit 2 Database Design

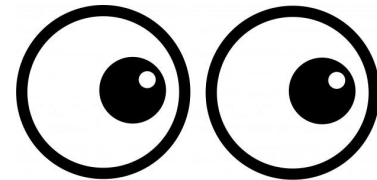
Index UNIT 2

1. Conceptual Data Model (2-1)
2. Logical Data Model (2-2)
3. Physical Data Model (3)



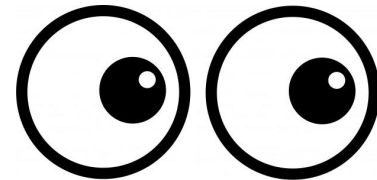
Index UNIT 2-1 Conceptual Data Model

1. Data modelling
2. Entity Relationship Diagram (ERD)
 - a. Entities
 - b. Attributes
 - c. Relationships
3. Extended Entity-Relationship (EE-R) Diagram
 - a. Superclass
 - b. Subclass
 - c. Hierarchical relationships



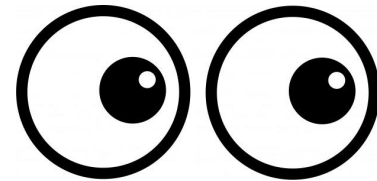
Index UNIT 2 - 2 (Logical Data Model)

1. From ERD to Relational Model
 - a. The Relational Model
 - b. Rules
 - c. Entity to table
 - d. Transforming Relationships
2. Normalization

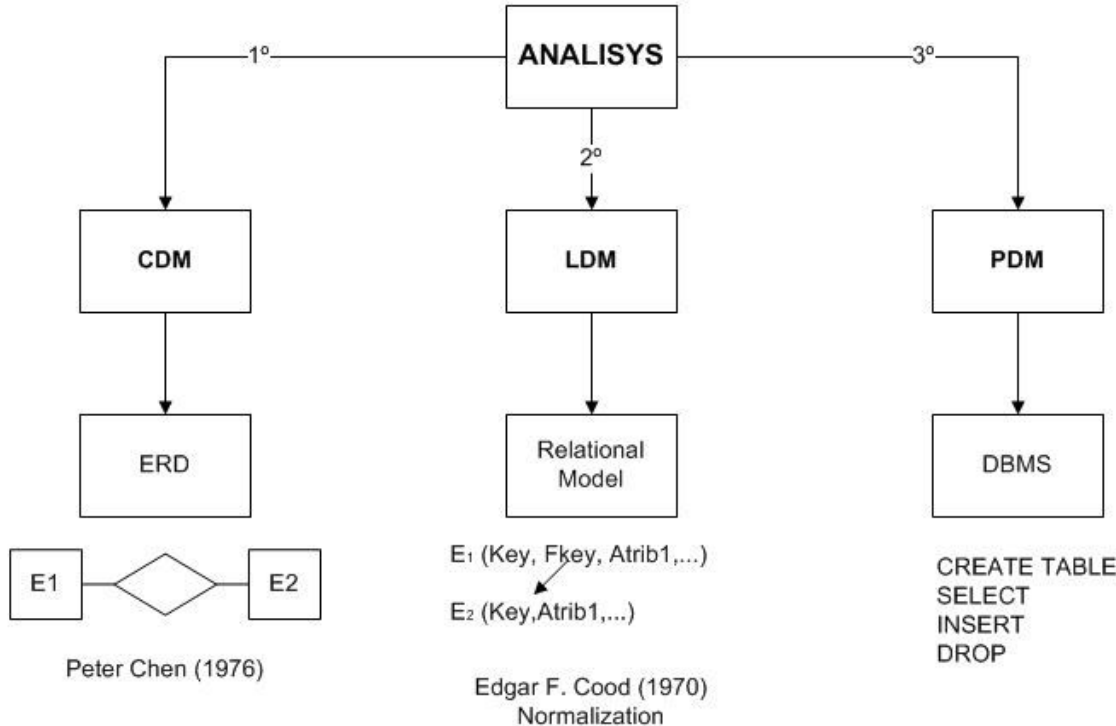


Index UNIT 3 (Physical Data Model)

1. Relational Databases Definition
2. Data model
3. Tables
4. Columns
5. Rows
6. Keys
7. Relationships
8. Normalization
- 9. SQL**
10. ACID Properties



Data modelling



DATABASE ANALISYS



CDM Conceptual Data Model



LDM Logical Data Model



PDM Physical Data Model



PRESENT YOUR SQL CODE

```

SELECT Surname
FROM Employee
WHERE WorkID = (SELECT WorkID
FROM Employee
WHERE Surname = "SMITH");
  
```

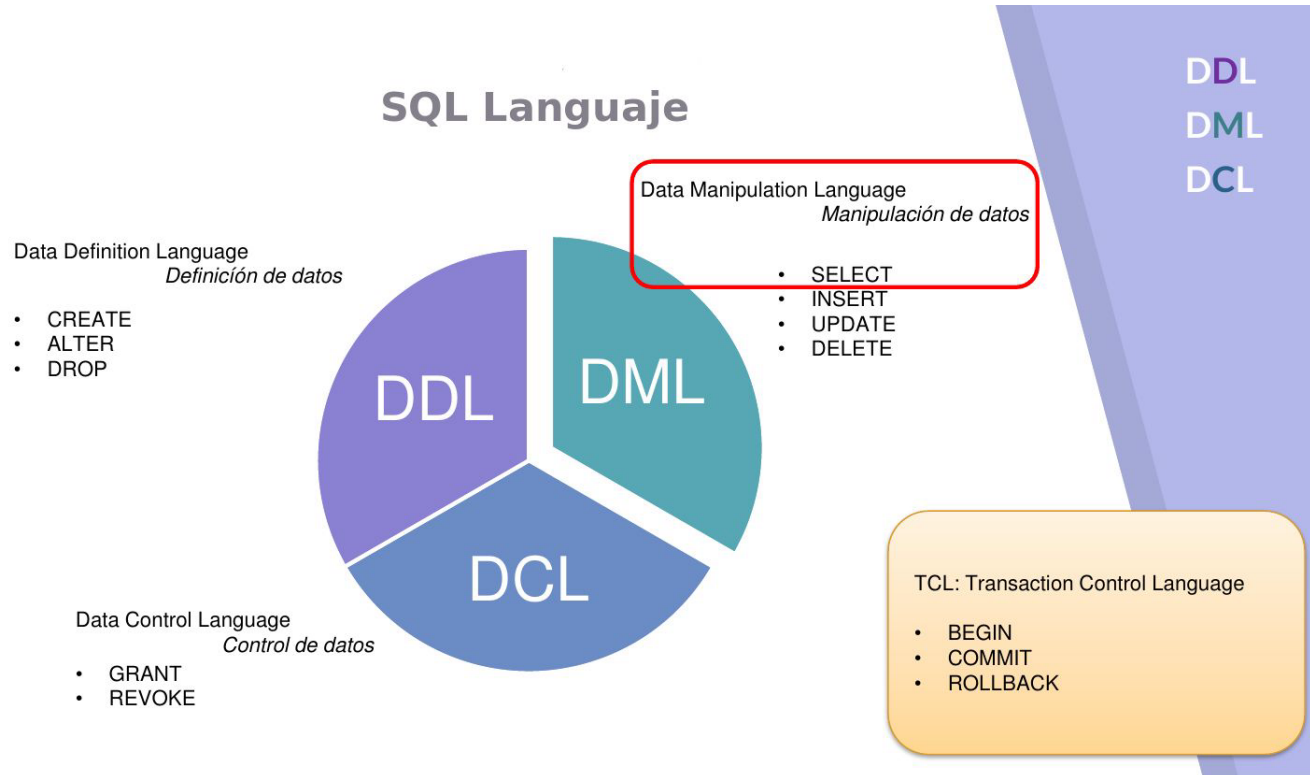
Data modelling

Each data solution must undergo a process that comprises the following phases:

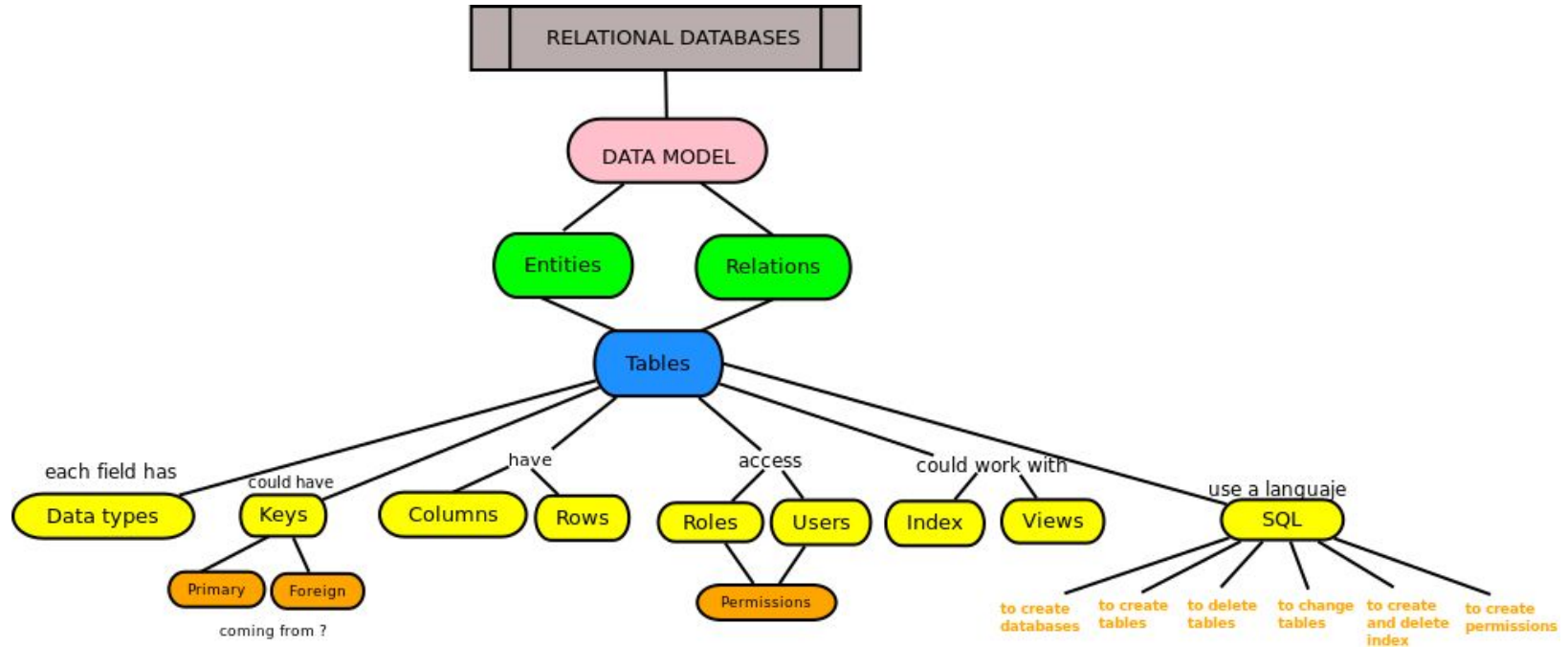
1. Definition of the **Conceptual Data Model**, using the **Entity/Relation Diagram** (ERD)
2. Through its graphical representation, the Entity/Relation Diagram (ERD), the conceptual data model becomes a **Logical Data Model**
3. In the software design process, the Logical Data Model is adapted to the **technical constraints** and **specification of the DBMS** (Data Base Management System) and the database, thus becoming a **Physical Data Model**
4. The Physical Data Model is **implemented in the DBMS using SQL** (DDL, DML, DCL)

The relevance of data modelling lies in the concept of independence: any data requisite should be taken into consideration regardless of the actual tools used in implementing the database.

SQL Structured Query Language



Entity Relationship Diagram (ERD)



Entity Relationship Diagram (ERD)

An entity relationship diagram (ERD) shows the **relationships of entity sets** stored in a database. It illustrates the **logical structure** of databases. An **entity** is an object or concept about which you want to store information.

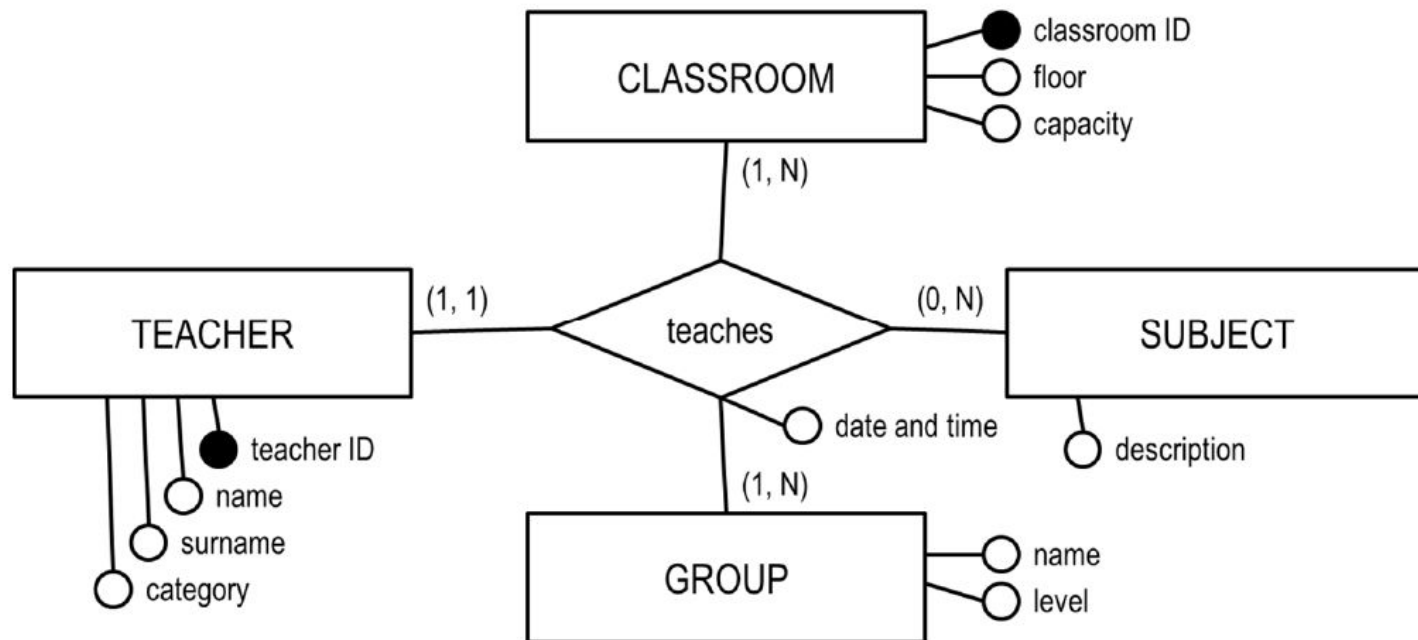
Peter Chen developed ERDs in 1976. Since then Charles Bachman and James Martin have added some slight refinements to the basic ERD principles.

At first glance an entity relationship diagram looks very much like a **flowchart**. It is the specialized symbols, and the meanings of those symbols, that make it unique.

An ER diagram is a means of visualizing how the information produced by a system is related. There are three **main components** of an ERD:

- **Entities**
- **Relationships**
- **Attributes**

Entity Relationship Diagram (ERD)



Entities

An **entity** is an **object or concept** about which you want to store information and is **distinguishable** from other objects. Example: the STUDENT Alberto Sánchez

An entity can be a **person, a place, an object, an event, or a concept** about which an organization wishes to maintain data.

The following are some examples of entities:

- Person: STUDENT, EMPLOYEE, CLIENT
- Object: COUCH, AIRPLANE, MACHINE
- Place: CITY, NATIONAL PARK, ROOM, WAREHOUSE
- Event: WAR, MARRIAGE, LEASE
- Concept: PROJECT, ACCOUNT, COURSE

Entities

They are represented by **rectangles**.



In the E-R diagram, we assign a name to each entity type. When assigning names to entity types, we follow certain naming conventions. An entity name should be a **concise singular noun** that captures the unique characteristics of the entity type.

Entities

It is important to understand the distinction between an entity type, an entity instance, and an entity set .

- An **entity type** defines a **collection of entities** that have same attributes.
- An **entity instance** (*ocurrencia*) is a **single item** in this collection.
- An **entity set** is a **set of entity instances**.

The following example will clarify this distinction: STUDENT is an entity type; a student (Alberto Sánchez) with ID number 555-55-5555 is an entity instance; and a collection of all students is an entity set (Pepe, Paco, Alex, Alan, Dereck)

Entity: singular noun

Entity Type : STUDENT

Entity Instance: Juan

Entity Set: Juan, Maria, Luis...

Strong entity

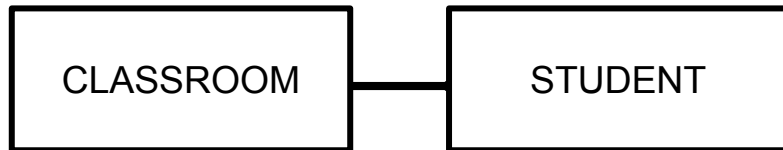
Weak entity

1 weak by existence: E

2 weak by identification

Entities. Strong

They are those that have an **existence by themselves**. That is, their existence does not depend on the existence of other entities.



In a school database, the existence of the entity CLASSROOM does not depend on the existence of instances of the entity STUDENT.

Entities. Weak (E)

weak entity

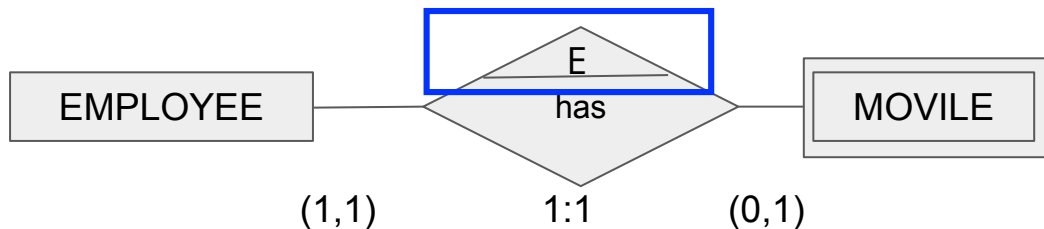
1

dependency in existence

will disappear if strong entity disappears

include E

Dependency in existence. If a strong entity instance disappears, weak entity instances will disappear. The representation of this type of dependency will include an E inside the weak relationship.



weak entity + surrogate key = becomes strong entity

weak entity + foreign key = still weak,
other strong entity sets its identifier

Entities. Weak

Strong entity → the entity set whose entities can be uniquely identified based on the values of the attributes of the entity set itself.

Weak entity → You need to add attributes in order to be able to identify each entity uniquely, either by adding a surrogate (substitute) key (in which case it becomes a strong entity) or by adding a foreign key (making it weak and the other entity set(s) its identifier(s)).

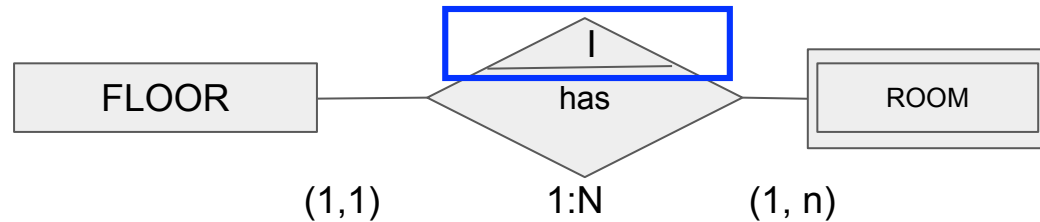
- An account of a particular bank has no existence if the bank doesn't exist anymore.
- A company insurance policy insures an employee and any dependents, the DEPENDENT cannot exist without the EMPLOYEE; that is, a person cannot get insurance coverage as a dependent unless the person is a dependent of an employee. DEPENDENT is the weak entity in the relationship "EMPLOYEE has DEPENDENT"

Entities. Weak (I)

weak entity

2 dependence in identification
cannot be identified on its own
need strong entity key

Dependence in identification. An instance of the weak entity cannot be identified on its own and must be identified by the key of the associated strong entity. It needs the strong entity key.



Both strong and weak entities are **usually named with singular nouns**.

Attributes

We represent an entity with a set of attributes. An **attribute** is a **property or characteristic of an entity type** that is of interest to an organization. Some attributes of common entity types include the following:

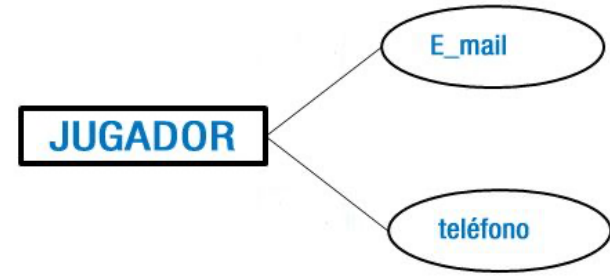
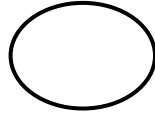
- STUDENT = {Student ID, SSN, Name, Address, Phone, Email, DOB}
- ORDER = {Order ID, Date of Order, Amount of Order}
- ACCOUNT = {Account Number, Account Type, Date Opened, Balance}
- CITY = {City Name, State, Population}

We use the following conventions while naming attributes:

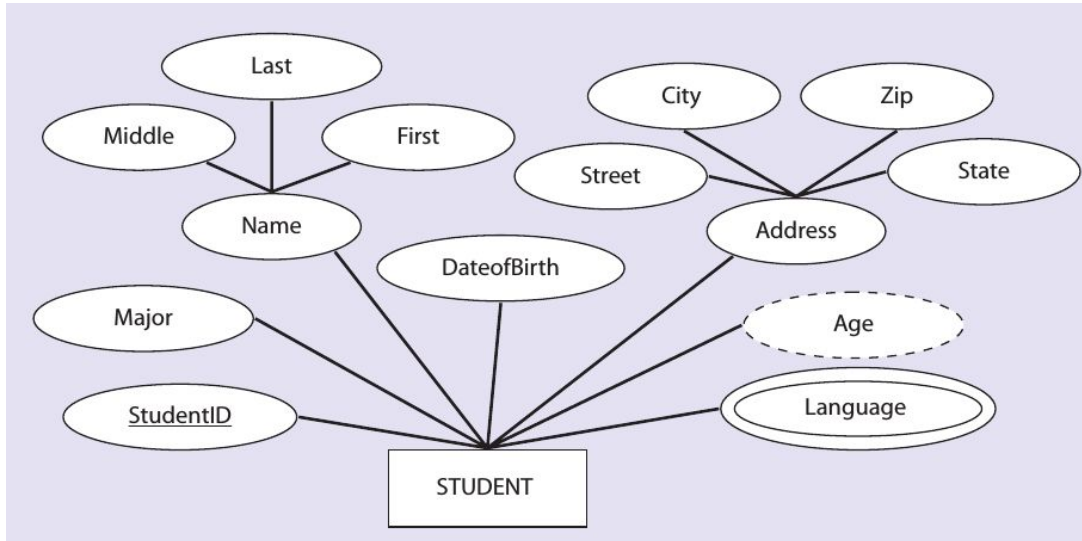
1. Each word in a name starts with an **uppercase letter followed by lower case letters**.
2. If an attribute name contains two or more words, the **first letter of each subsequent word is also in uppercase**, unless it is an article or preposition, such as "a," "the," "of," or "about"

Attributes

They are represented by ovals.



There are several types of attributes featured in this figure: simple, composite, single valued, multi-valued, stored, and derived attributes.



Attribute

1 single

2 multivalue

1 simple/atomic

2 composite

3 derived, stored: caculated

1 mandatory

2 optional

Attributes. Types

Single. It has a single value for each entity instance.

An example of this type of attribute is a person's identity document.

In the figure: a student has only one date of birth and one student identification number.

Multivalued. Can take **different values** for each instance of entity.

For example, it may be necessary to store a set of telephone numbers where a student can be contacted (landline, mobile, etc), but also the case where he/she does not have a telephone consider the case where you do not have a telephone (you may not want to provide it).

In the figure: Languages stores the names of the languages that a student speaks.

Attributes. Types

Simple or atomic attribute. An attribute that cannot be divided into other parts or attributes, it has a single element. It is not possible to extract from this attribute smaller parts that can have meaning.

An example of this type of attribute could be City and State

Composite attribute. These are attributes that can be divided into parts, these will constitute other attributes with their own meaning.

Example, the student's address could be considered as a composite attribute consisting of the composed of the City, State, Zip, Street.

Attributes. Types

Derived or stored attributes. The value of this type of attribute can be derived from the value(s) of other related attributes.

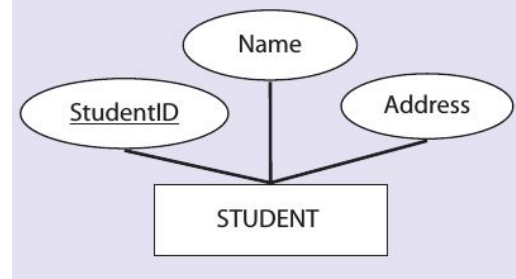
A classic example of a derived attribute is age, if the date of birth has been stored in some attribute, the age is a value that can be calculated from that date and the current date.

Attributes. Types

Mandatory attribute. It is one that must always be defined for an entity or relationship. Example, for the entity STUDENT it will be necessary to have some attribute that identifies each instance of the entity, such as its ID. A key is a mandatory attribute.

Optional attribute. It is one that could be defined or not for the entity. That is, there may be entity instance for which this attribute is not defined or has no value.

Attributes. Key



A **key attribute (or identifier)** is a single attribute or a combination of attributes that **uniquely identify an individual instance of an entity** type.

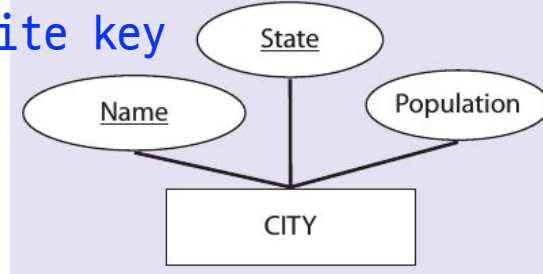
No two instances within an entity set can have the same key attribute value.

- For the STUDENT entity shown before, StudentID is the key attribute since each student identification number is unique. Name, by contrast, cannot be an identifier because two students can have the same name.
- For EMPLOYEE, the employee's social security number might be the employee's key attribute.

We **underline** key attributes in ERD.

Attributes. Key

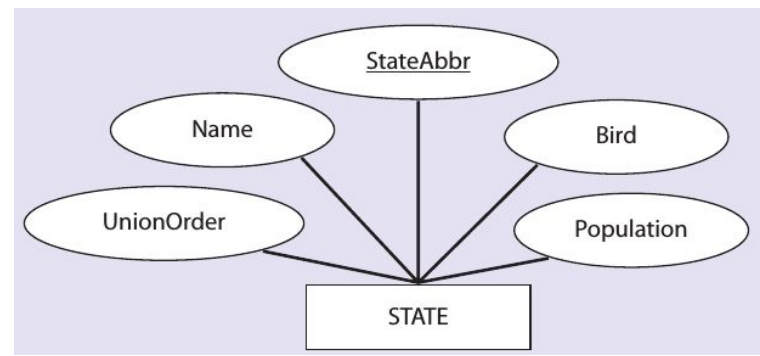
composite key



Sometimes no single attribute can uniquely identify an instance of an entity type. However, in these circumstances, we identify a set of attributes that, when combined, is unique for each entity instance. In this case the key attribute, also known as **composite key**, is not a simple attribute, but a **composite attribute that uniquely identifies each entity instance**.

For example, consider the CITY entity type. This category includes, potentially, all the cities in the United States. Notice that none of the attributes (i.e. Name, State or Population) can serve as a key attribute since there are many cities in each state and two cities could possibly have the same name or population. However, the composite attribute {Name, State} is a valid key attribute for the CITY entity as no two cities within a state can have the same name.

Attributes. Key



An entity **can have more than one attribute that qualifies to be an identifier**. For the entity shown in this figure, each of the attributes Name, StateAbbr, and UnionOrder (the order in which the state entered the union of the United States) can be an identifier. In this case, it is a matter of **preference** as to which attribute is made an identifier or key attribute (see **candidate key**).

Attributes. Key

The different types of database keys are:

- **Natural key**: an attribute that can uniquely identify a row, and exists in the real world or is used for business.
- **Primary key**: the single or composite attribute that can uniquely identify a row.
- **Candidate key**: an attribute that could be the primary key.
- **Surrogate key**: an attribute that can uniquely identify a row, and does not exist in the real world.
- **Alternate key**: a candidate key that is not the primary key.
- **Foreign key**: an attribute that is used to refer to another record in another table.

Attributes. Key

1. 모든 key는 uniquely identify할 수 있어야 한다
2. natural key도 instance를 특정할 수 있어야 함

Example.



For the entity STUDENT the attribute name cannot be a natural key because they do not uniquely identify the entity (it may be the case that two persons have the same name).

However, the attributes StudentID and Address would be two suitable **candidate** keys. If we choose StudentID as the primary key, Address would be the **alternative key**.

Attributes. Key



Let's look at this example – an employee record.

Employee Number: 3250

Passport ID: 5655555

First Name: Ben

Last Name: Brumm

Address: 897 Main St, London, VIC, UK 3000

Email: ben@company.com

Phone: 44 12 345 678

Social Securty Number: 28002123456

Attributes. Key

- A **natural key** is a key that can be defined naturally. In other words, it does not require adding an additional column or columns. For example, assume a user table that stores usernames and emails. As these columns help to identify the rows uniquely, these are **primary keys**. Furthermore, they were already available in the table. Therefore, they are natural keys.
- **Primary key** refers to the minimal set of columns that helps to **identify a record in the table uniquely**. The primary key has a few characteristics. Mainly **1**, two rows cannot have the same primary key value. Moreover, each row should have a value for the primary key. **2**, the primary key value cannot be null.
- Moreover, a surrogate key is a primary key, while the primary key is a **candidate key**.
- Unlike a primary key, a **surrogate key is a made-up primary key**. In other words, it is a **column added** to the table. We can use a surrogate key when there are no columns to define as a natural primary key. These keys are unique as we create it when there is no other natural primary key. They are meaningless. They serve no purpose, except to technically identify a record uniquely in one source system.

Attributes. Domain of an attribute

Set of allowed values for an attribute. The attributes take values from one or more domains.

All possible values that an attribute can take shall be within the domain. Several attributes may be defined within the same domain.

Example, the attributes first name, last name of the entity STUDENT, are defined within the domain of character strings of a **certain length**. It is good to set appropriate limits to the domains so that the DBMS performs a verification of the data to be stored.

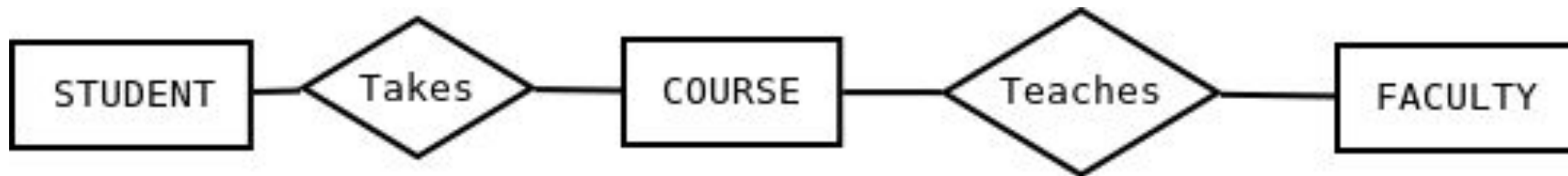


Domain of the DateofBirth, StudentID??

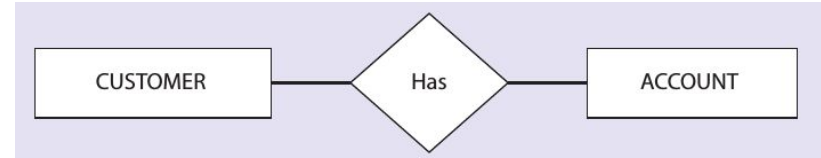
Relationships

Entities in an organization do not exist in isolation but are related to each other.

- Students take courses and each STUDENT entity is related to the COURSE entity.
- Faculty members teach courses and each FACULTY entity is also related to the COURSE entity. Consequently, the STUDENT entity is related to the FACULTY entity through the COURSE entity. E-R diagrams can also illustrate relationships between entities.
- We define a **relationship** as an **association among several entities**. They are represented by **diamond shapes** and take the form of **present tense verb** for describing them.



Relationships



Consider, for example, an association between customers of a bank.

If customer Williams has a bank account number 523, then the quality of ownership constitutes a relationship instance that associates the CUSTOMER instance Williams with the ACCOUNT instance 523.

We can think of the **relationship instance** as a verb that links a subject and an object:

- customer Williams has an account
- student John registers for a course
- professor Smith teaches a course

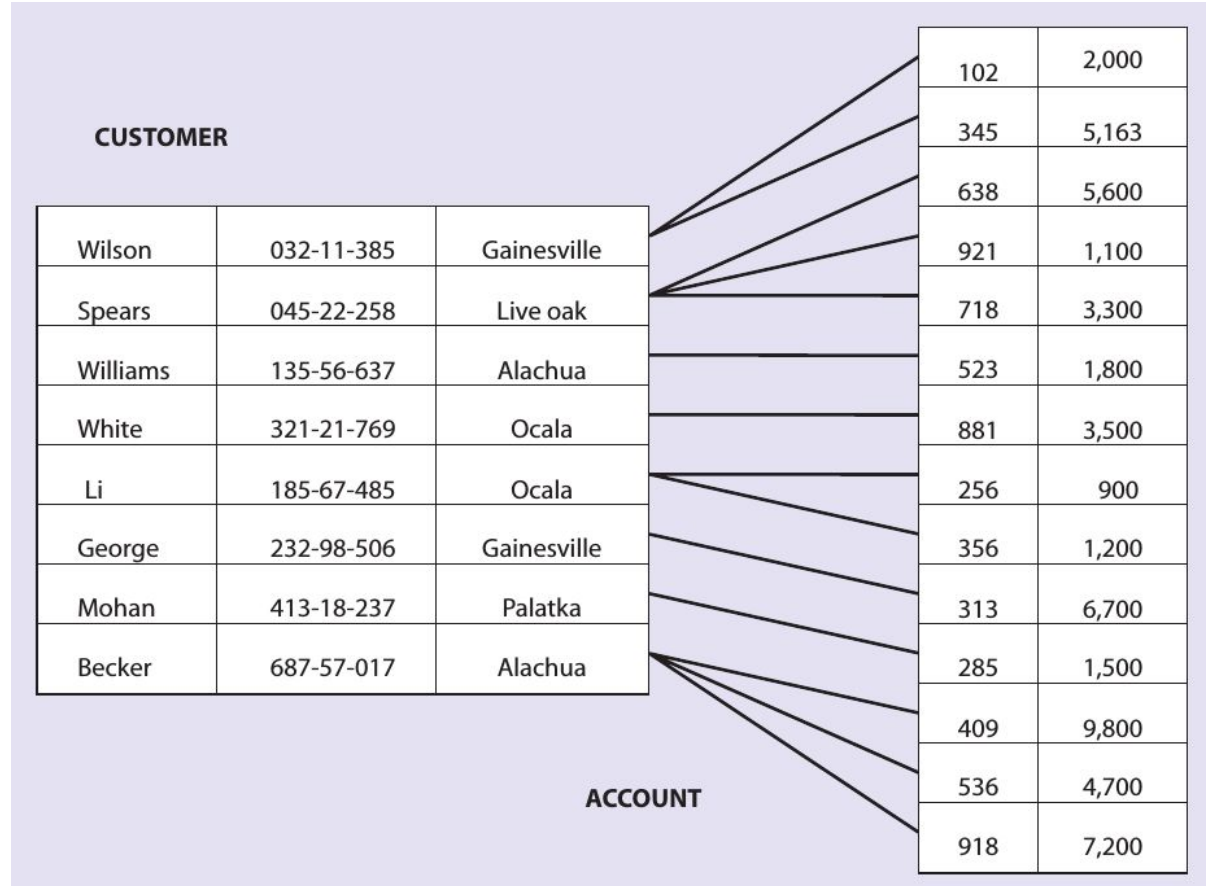
✓ **relationship set:**
all matching relationship instances

✓ **relationship type:**
relationship between entity types

A **relationship set** is a grouping of all matching relationship instances and the term **relationship type** refers to the relationship between entity types.

Relationships

For example, this figure illustrates a **relationship set** between the CUSTOMER and the ACCOUNT instances.



Relationships



1. Define the terms entity, attribute and relationship.

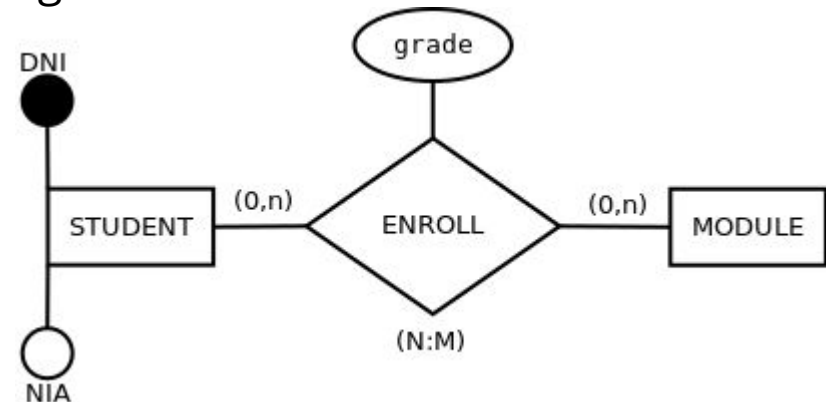
Relationships. Attributes

Relationship can also have attributes that describe them. It depends on the entities participating in the relationship.

Example. Let us consider the relationship Enrolls between the entities STUDENT and MODULE. We could associate to the relation Enrolls a **grade attribute** to specify the grade obtained by a STUDENT in a given MODULE.

The representation of primary keys in the E/R model can be done in two ways:

- If ellipses are used to represent the attributes, the attributes that form the primary key shall be **underlined**.
- If circles are used to represent attributes, a **black circle** shall be used for those attributes that form the primary key.

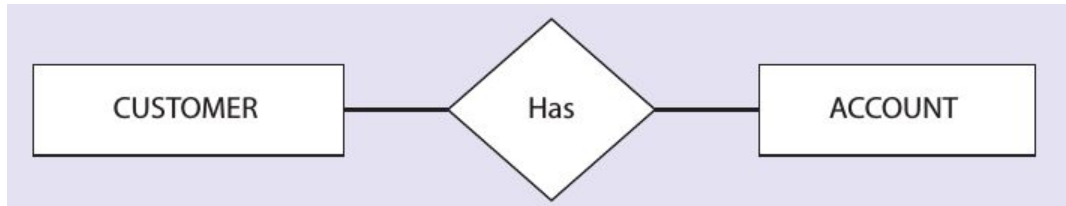


Relationships. Grade

grade 1 = unary = reflexive/recursive
grade 2 = binary
grade 3 = n-ary, degree n
double grade

The number of entity sets that participate in a relationship is called the **grade** of relationship.

For example, the grade of the relationship featured in this figure is two because CUSTOMER and ACCOUNT are two separate entity types that participate in the relationship.

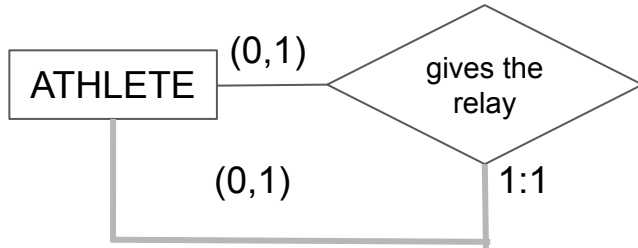


The three most common grades of a relationship in a database are unary (grade 1), binary (grade 2), and ternary (grade 3).

Relationships. Grade 1. Reflexive/Recursive/Unary

Number of participating entities: grade 1, grade 2, grade 3, ...

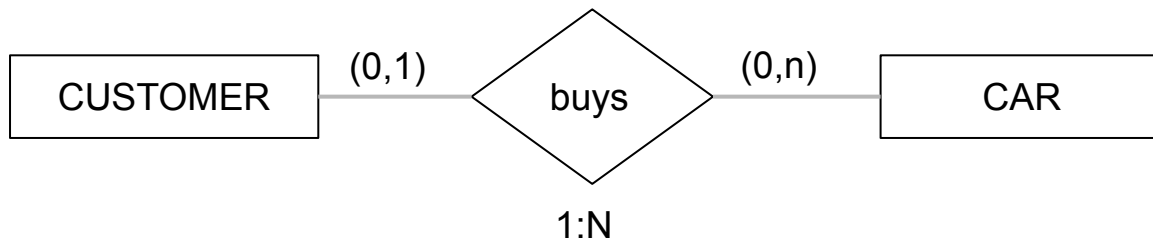
Unary (self-linked) or grade one. A single entity is involved. Also called reflexive or recursive.



Relationships. Grade 2

Number of participating entities: grade 1, grade 2, grade 3, ...

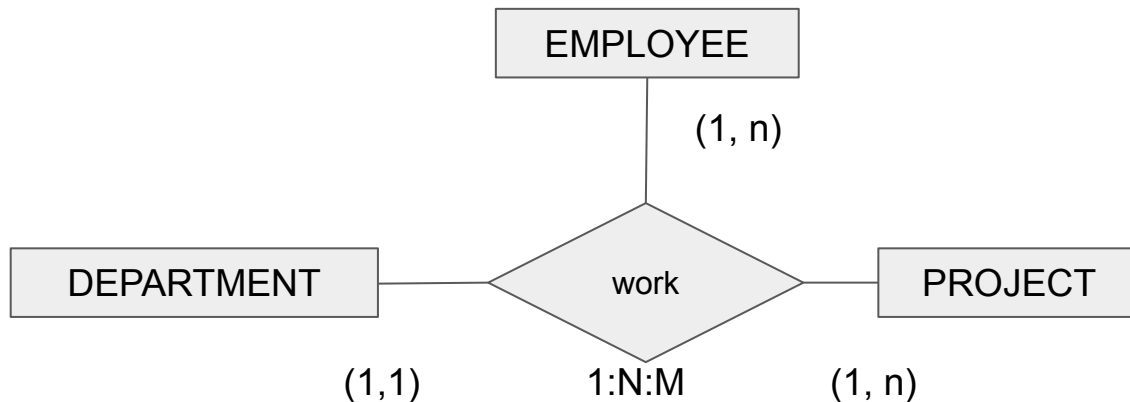
Binary or degree two. Two entities are involved. In general, the conceptual scheme of the database will seek to have only this type of relationship.



Relationships. Grade 3

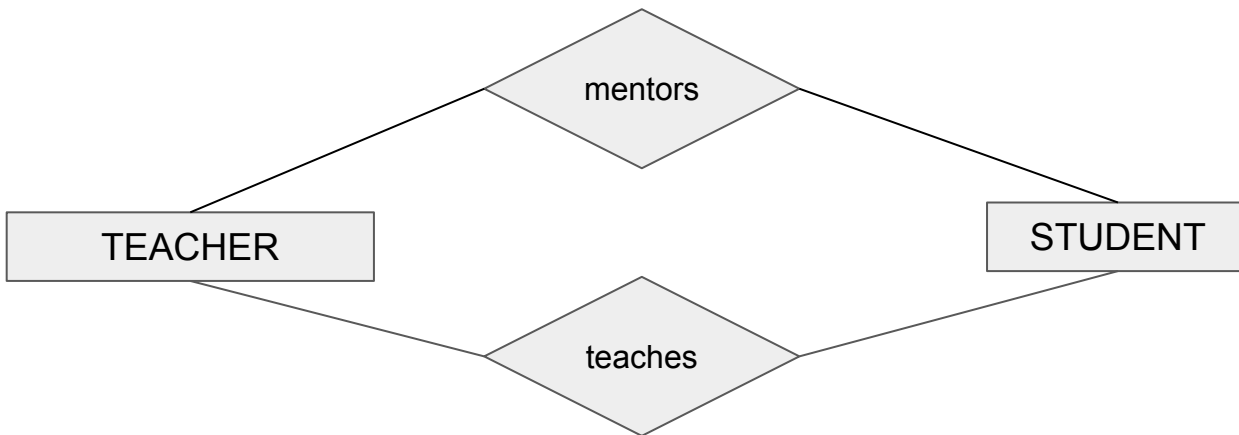
Number of participating entities: grade 1, grade 2, grade 3, ...

N-ary or degree n. Involves n entities. This type of relationships are unusual and should be simplified to lower degree relationships.



Relationships. Double Grade

Two entities can be related through two relationships.



Relationships. Cardinality

Relationship cardinality: specifies the **maximum** number of instances of an entity that may participate in one **relationship** instance. (X:X)

Entity cardinality: **minimum and maximum** number of instances of an entity that may relate to an instance of another **entity**. (X,X)

We find the classic situation of a snake biting its tail.

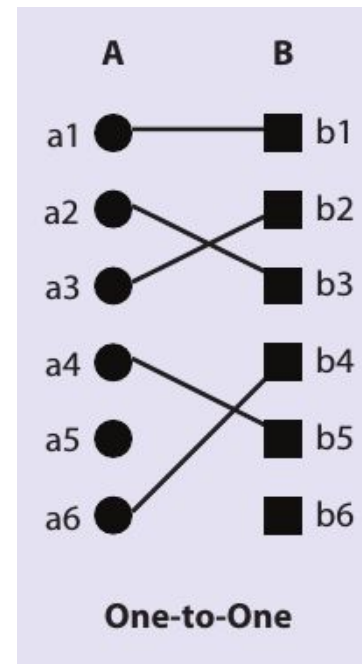
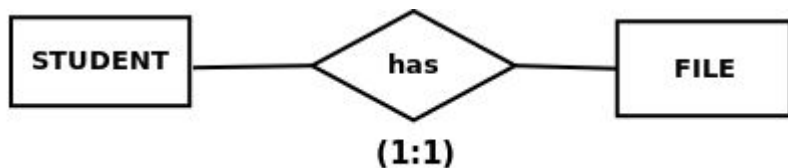
Relationships. Relationships Cardinality (X:X)

Specifies the **maximum** number of instances of an entity that may participate in one relationship instance. Depending on the number of instances of every entity, it can exist:

1:1 (one to one) relationship

An instance of one entity is related to one and only one instance of another entity.

Example: for each instance of the entity STUDENT there will only be one related instance of the entity FILE (academic record) and vice versa. A STUDENT has an associated FILE and a FILE belongs to one STUDENT.

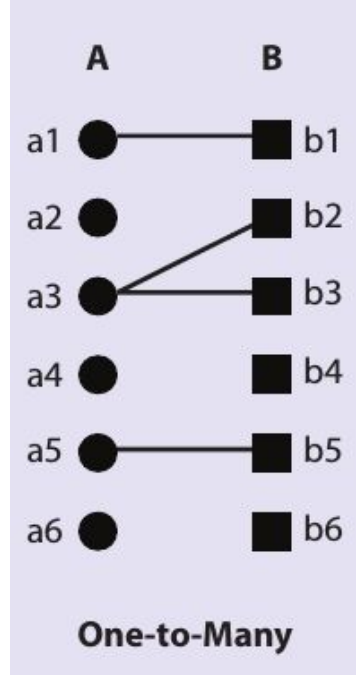
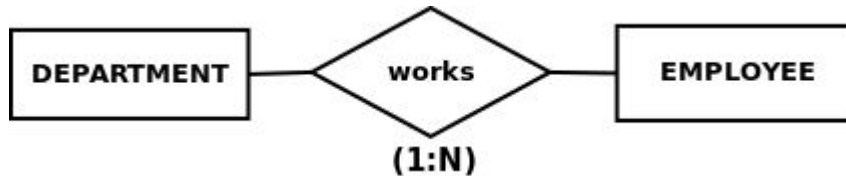


Relationships. Relationships Cardinality (X:X)

1:N (one to N / one to many) relationship

An instance of one entity can be related to several instances of another entity, but each instance of the second entity can only be related to a single instance of the first entity.

Example, for each instance of the DEPARTMENT entity, there may be several instances of the EMPLOYEE entity and for the EMPLOYEE entity there will only be one related instance of the DEPARTMENT entity. In a DEPARTMENT there may be several EMPLOYEES, but an EMPLOYEE will only belong to one DEPARTMENT.

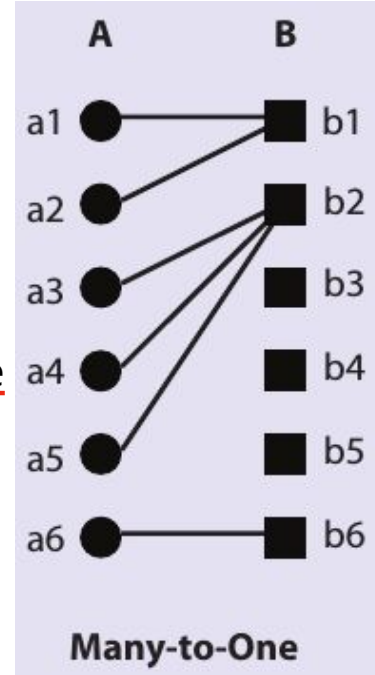
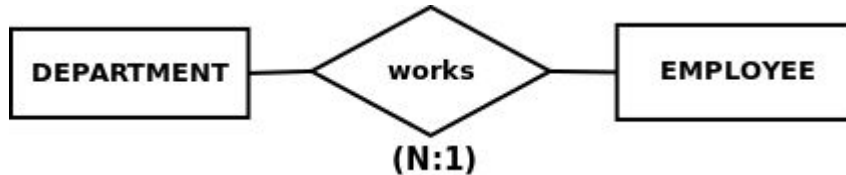


Relationships. Relationships Cardinality (X:X)

N:1 (N to one / many to one) relationship

An instance of one entity can be related to several instances of another entity, but each instance of the second entity can only be related to a single instance of the first entity.

Example, for each instance of the DEPARTMENT entity, there may only one instance of the EMPLOYEE entity and for the EMPLOYEE entity there will be several related instances of the entity DEPARTMENT. In a DEPARTMENT will be only one EMPLOYEE, but an EMPLOYEE can works for more than one DEPARTMENT.

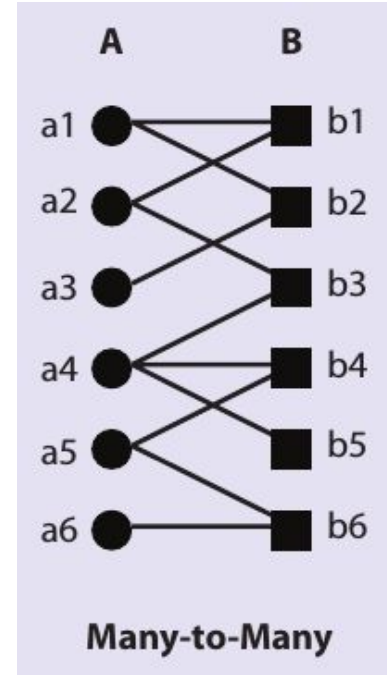
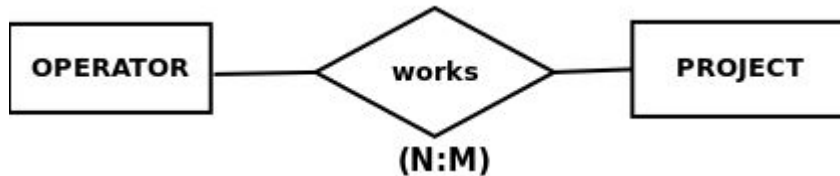


Relationships. Relationships Cardinality (X:X)

N:M (N to M / many to many) relationship

Each instance of an entity can be related to several of another entity, and each instance of the second entity can also be related to several instances of the first.

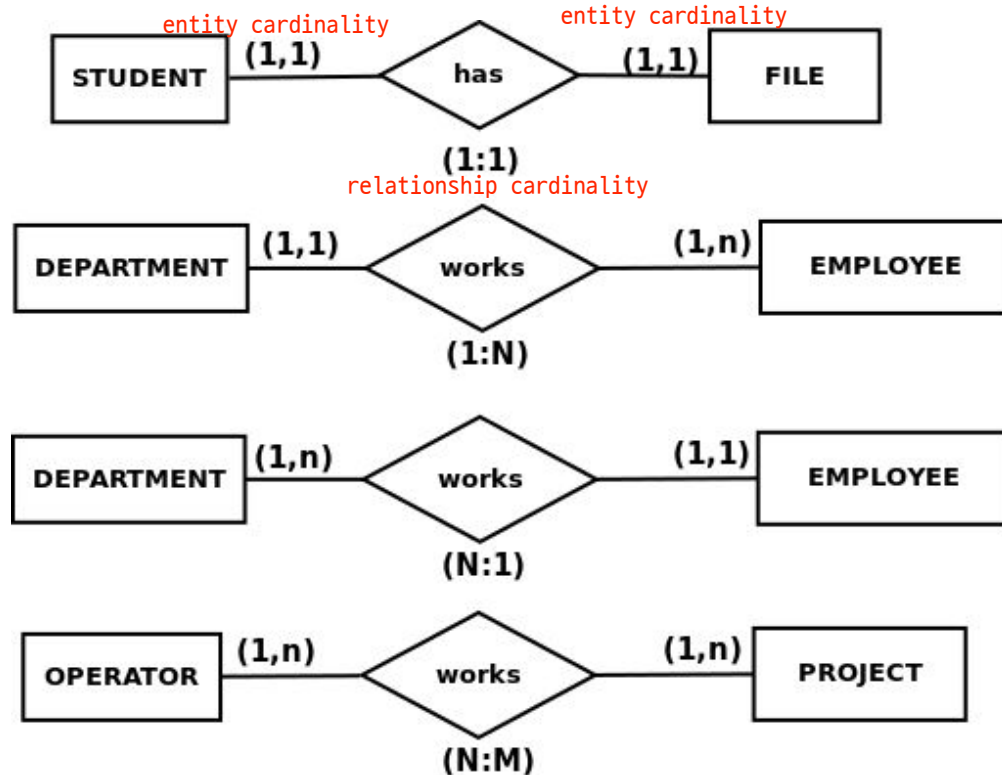
Example, an instance of the entity OPERATOR is related to several entities of the entity PROJECT (an operator works on several projects), and a PROJECT entity may be related to several instances of OPERATOR (there may be several operators in a project).



Relationships. Entities Cardinality (x,x)

It defines the **minimum and maximum** number of instances of an entity that may be related to an instance of another or other entities, identifying optional relationships (in which there doesn't have to be correspondence).

The **entity cardinality** is indicated on **both sides** of the relation. Its maximum value matches with the value of the cardinality corresponding to the side of the relation in which we find ourselves.



Relationships. Entities Cardinality (x,x)

The cardinality with which an entity **participates** in a relationship specifies the **minimum and maximum** number of instances in which each instance of that entity may participate. It can be:

0..1 (zero to one). Each instance of the entity can relate to an instance of the second entity or not. It can not relate to several.

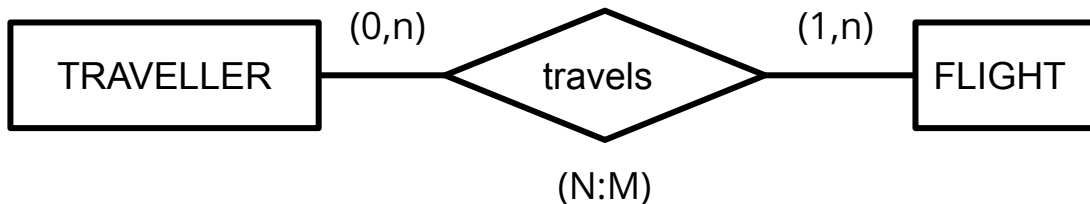
1..1 (one to one). Each instance of the entity must necessarily relate to one and only one instance of the second entity.

0..n (zero to many). Each instance of the entity has not limited its relation to instances of the second entity. It can be related to one, several or none.

1..n (one to many). Each instance of the entity must necessarily relate to at least one instance of the second entity. It can relate to several.

Relationships. Entities and Relationships Cardinality

Example. Let's imagine an airline database. At least one TRAVELLER has flown on at least one FLIGHT (1,n) and on one FLIGHT several, or none, TRAVELLERS have flown (0,n).



Relationships. Entities and Relationships Cardinality



Define, grade of relationships and cardinality of relationships and entities

Relationships. Examples



Create the ERD for next statements:

1. A person cannot be married, but only can be married to a person. The Date of marriage is an attribute of this relationship.
2. An employee supervises many employees but a supervisee can have only one supervisor.
3. A part can use many parts and can be used in the making many parts.
4. An employee can manage at most one department and each department is managed by at most one employee.
5. A customer can place several orders or may choose not to order at all, each order must be placed by exactly one customer.
6. Each student can register in many courses, a single course, or no course at all.
7. An employee works only for a department; on a department can work more than one employee.
8. An employee works only for a department; on a department can work more than one employee. Employees work for departments on projects.
9. Students use equipments to work on projects.
10. In a competition, musicians perform individually and in small groups for judges who rate the performances. Each performance requires an artist (musicians), a composition, and a venue.

Exercises (I)



- 1.- Draw up a list of possible entities for a bar cash register management application.
- 2.- Draw up a list of possible relationships for the bar cash register management application.
- 3.- Complete the previous exercise with entity cardinality.
- 4.- Represents the ERD for the entities, relationships from previous exercises.
- 5.- Customers of our video club can rent films and each film can be rented to several customers.
- 6, 7, 8, 9, 10, 11, 12 ... See below

Exercises (I)



6.- A company dedicated to the manufacture, sale and installation of made-to-measure bedrooms would like to create a database where the sales and assemblies of the bedrooms are reflected. For this purpose, the following aspects are taken into account the following aspects:

1. Each **bedroom** must be set up by at least two **fitters**.
2. The same fitter may fit several bedrooms.
3. For each bedroom, given that it is tailor-made, we are interested in its code, name and price.
4. The same fitter may have fitted different bedrooms on different dates. We are interested in knowing each assembly date.
5. We are interested in the customer's VAT, NIF, name, address
6. We are interested in the fitter's VAT number, name, address and telephone number.
7. Each bedroom is bought by a client and the same client can buy one or more bedrooms, each one on a specific date.

Exercises (I)



7.- Each division of our company is composed of several departments. Each department has many employees assigned, but each employee only works for one department. Each department is headed by an employee, and no employee can head more than one department.

Exercises (I)

7.1.- Add the following requirement: each employee has another employee as a manager.



Exercises



8.- It is desired to store information about the metro network, knowing that the **trains** are **assigned** to a certain **line**, each one (**train**) is **stored** in a certain **garage** (which is in a station), and on each line each **station** has an order number.

A train can be assigned to only to one line on the same date.

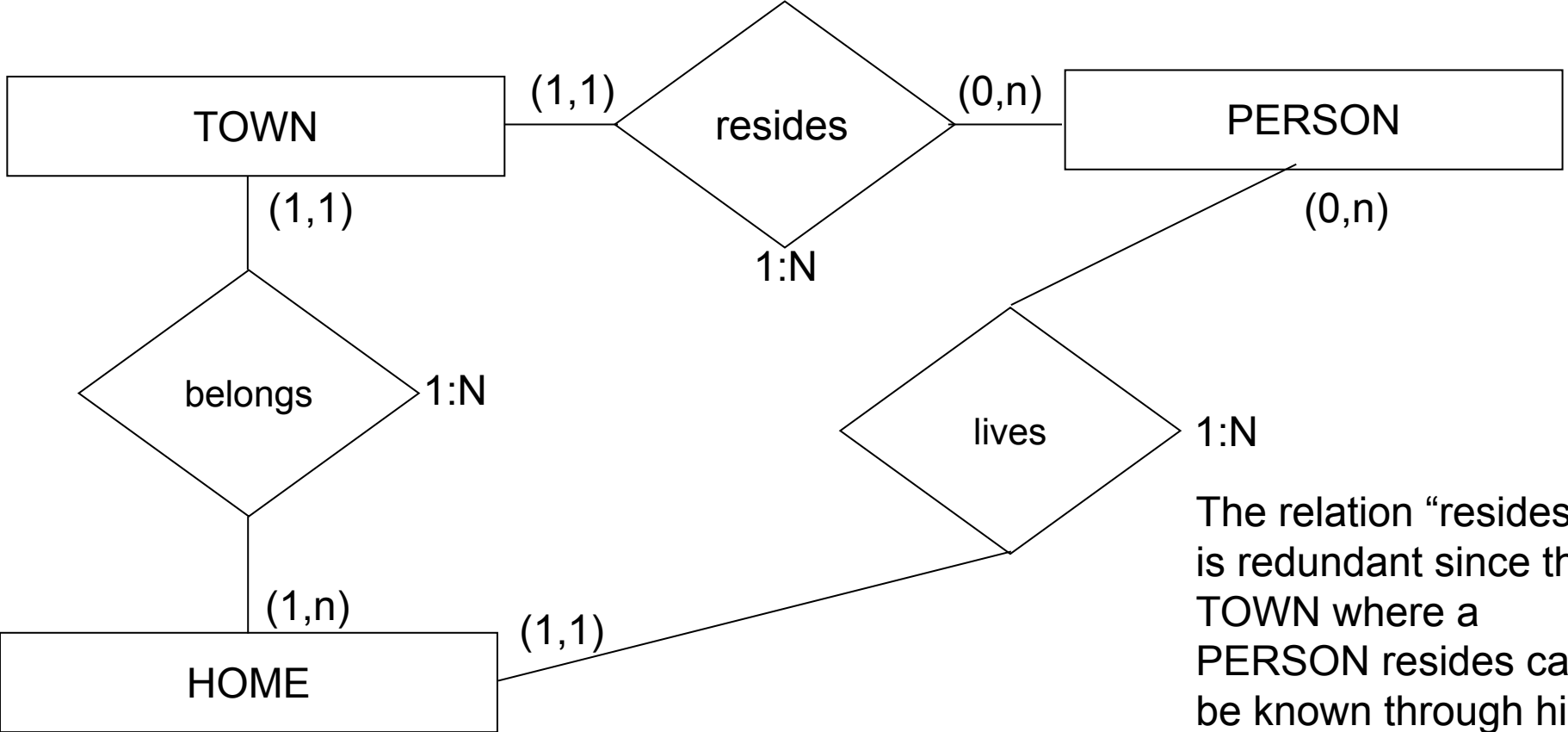
If there are works on a line trains can't go though.

Not all stations have have garages.

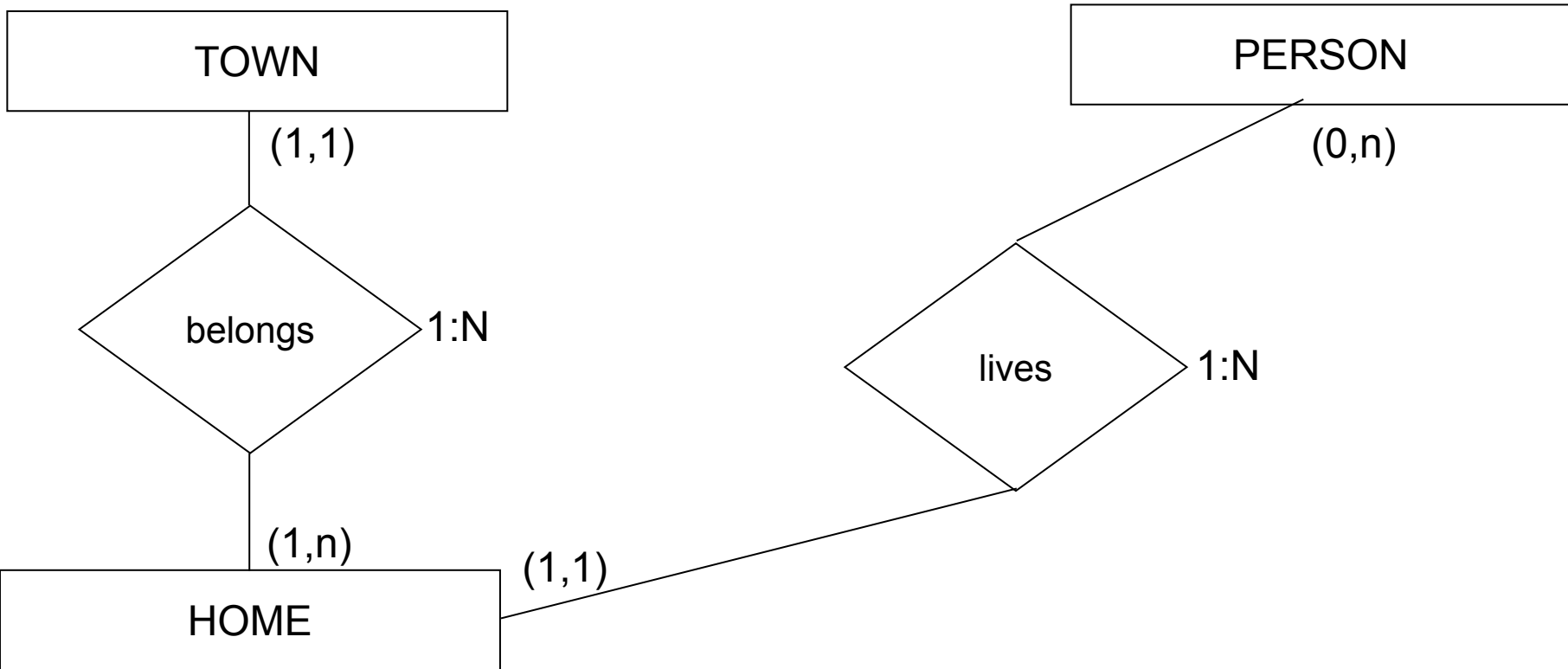
Redundancy control has loop!

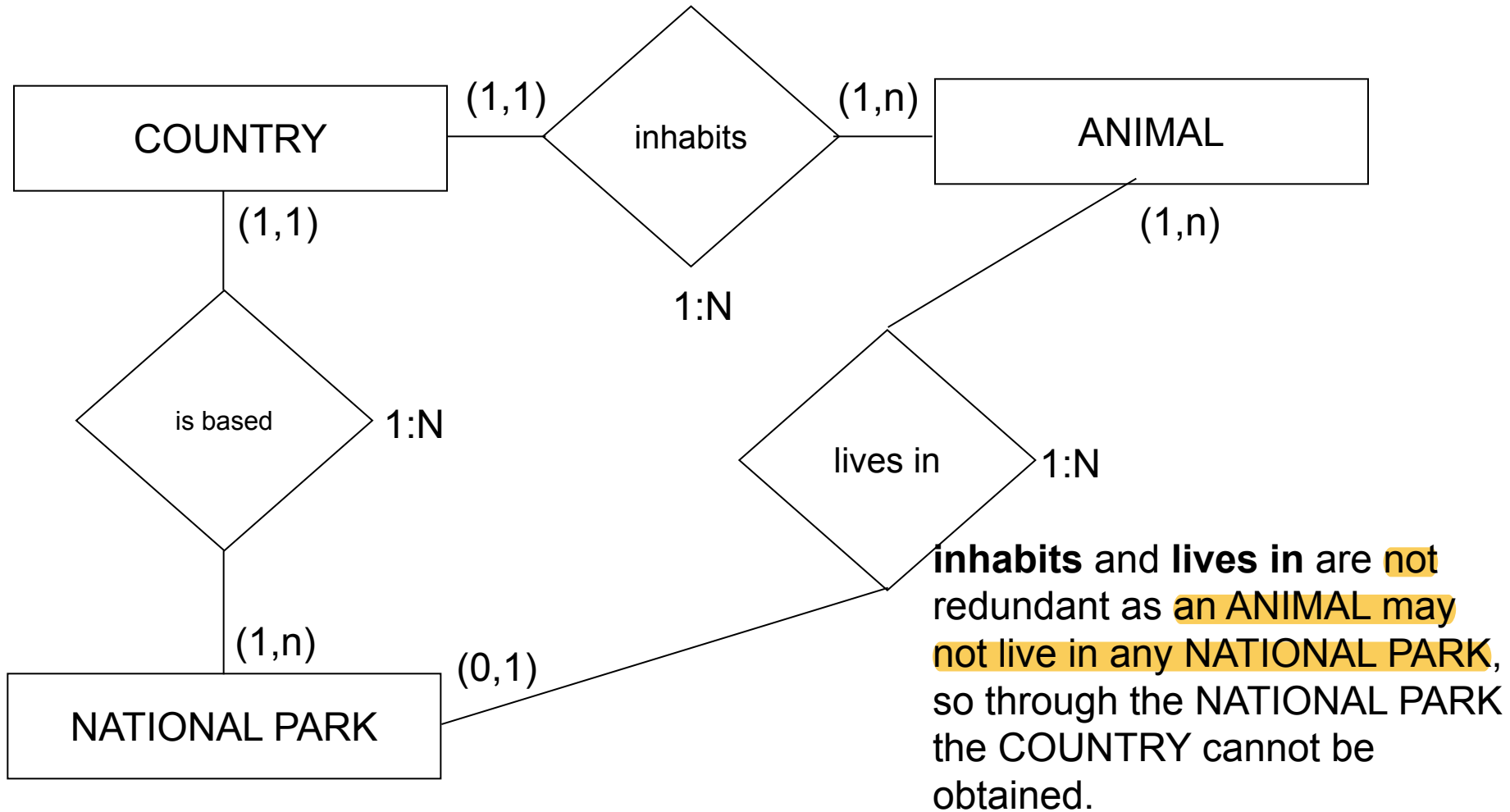
A schema is redundant if no information is lost by removing an element from the schema. The data of that element can be deduced from the other elements.

A redundant schema contains a loop.



The relation “resides” is redundant since the TOWN where a PERSON resides can be known through his or her HOME.





Exercises (I)



9.- A hardware store sells different **products**. Each one of the products can be created by several **manufacturers**. The price of each product will vary depending on its manufacturer. Each time a **customer** purchases one or several products, a list (**invoice**) is generated (**issued for**), that (**this**) includes the date, products purchased, quantity of each product, price, total price and taxes.

Exercises (I)



10.- The data model of a library has to be defined with the following basic constraints.

1. Each registered **book** has several physical **copies**.
2. Each book corresponds to one or more subjects (**THEME**), is written by one or more **authors** and published by a single publisher.
3. Copies are lent to members (**CLIENT**). The date of the loan must be kept.
4. It is necessary to store the nationality or nationalities of each author, as well as the country where each **publisher** has its headquarters.

Exercises (I)



11.- Modify the above diagram to meet the following requirements:

- The ISBN (International Standard Book Number) of each book must be stored.
- Each book will only correspond to a specific subject.
- Instead of storing the country where the publisher's head office is located, all countries in which each publisher has offices will be stored, as well as the office address.

Exercises (I)

12.- A hospital has a **patient** register, a **staff** register and a **floor** register. In addition, a register is kept with the staff working and the patients admitted to each floor.

1. From the staff we are interested in the **employee** number, name, address and telephone number. Each employee has a unique number.
2. From the patients we are interested in the registration number (assigned to them when they are admitted) and the name.
3. From the floor we are interested in the name and the number of beds it has.
4. It is also known that an employee works in only one floor and that several employees work in one floor. The same applies to patients.

References

- <https://stackoverflow.com/questions/4741967/example-of-a-strong-and-weak-entity-types>
- <https://pediaa.com/what-is-the-difference-between-surrogate-key-and-primary-key/>
- <https://ikastaroak.birt.eus/edu/argitalpen/backupa/20200331/1920k/es/DAMDAW/BD/BD02/es DAMDAW BD02 Contenidos/website index.html>
- <https://ikastaroak.birt.eus/edu/argitalpen/backupa/20200331/1920k/es/DAMDAW/BD/BD02/es DAMDAW BD02 Contenidos/website 41 tipos de atributos.html>
- http://dia-installer.de/howto/create_shape/index.html.en (use F9 to add shape)
- <https://pdfcoffee.com/er-diagram-3-pdf-free.html>

TOOLS TO DRAW ER-Diagrams

- <https://app.creately.com/> (on line)
- DIA (installed in Linux)
- Lucid → <https://www.lucidchart.com/> (on line)

Extended Entity-Relationship (EE-R) Model

Some authors added restrictions to the entity relationship diagram that, in some cases, expanded its semantic magnitude. Although there is no unanimity on the need and suitability of these additions or on the notation to be used, these are some of the most recognizable elements of the **extended entity relationship diagram**.

- **Superclass (Supertype)**, is an entity type that has one or more distinct subgroups with unique attributes.
- **Subclass (Subtype)**, is an entity type that shares common attributes or relationships distinct from other subclasses.
- **Hierarchical relationships** (IS A) that occur when one entity can be subdivided into other entities.
 - **Heritage**. Represents a relationship between classes in which an entity is composed of one or more entities.
 - **Exclusiveness (XOR)**. The existence of a relationship prevents the existence of another. Suppose an educational center where a teacher can teach or work on a research project, but not do both at the same time.
- **Aggregation**. Instances of several entities represent the parts of a whole represented in the instances of another entity. **combining things into a single group/total**

Extended Entity-Relationship (EE-R) Model

- There are several reasons for introducing superclass and subclasses into an E-R model.
 - First, incorporating a superclass and subclasses maintains the cleanliness of the structure and improves the conceptual flow.
 - Second, it adds familiar semantic information to an E-R diagram.
- Consider using subtypes if the following conditions apply:
 - 1) Certain attributes apply to some, but not all, instances of an entity type.
 - 2) An instance of a subtype participates in a relationship unique to that relationship.

Extended Entity-Relationship (EE-R) Model

What is desirable?

Good construction. Good Interpretation.

Complete. It is possible to verify that each of the requirements is represented in this diagram.

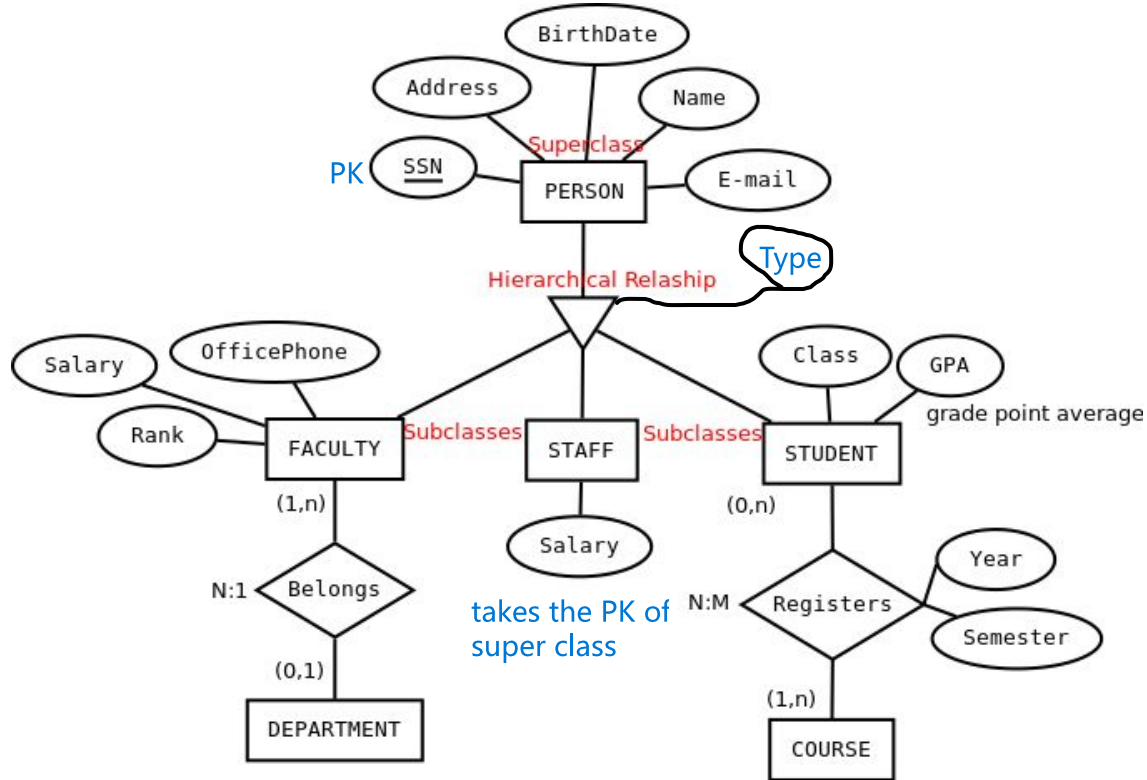
Correct: It uses all the elements of the Entity/Relationship model appropriately. The accuracy of a diagram can be analysed from two perspectives:

- Syntactic. No misrepresentations occur in the diagram.
- Semantic. The representations mean exactly what is stipulated in the requirements.

Minimal. No redundant elements and information.

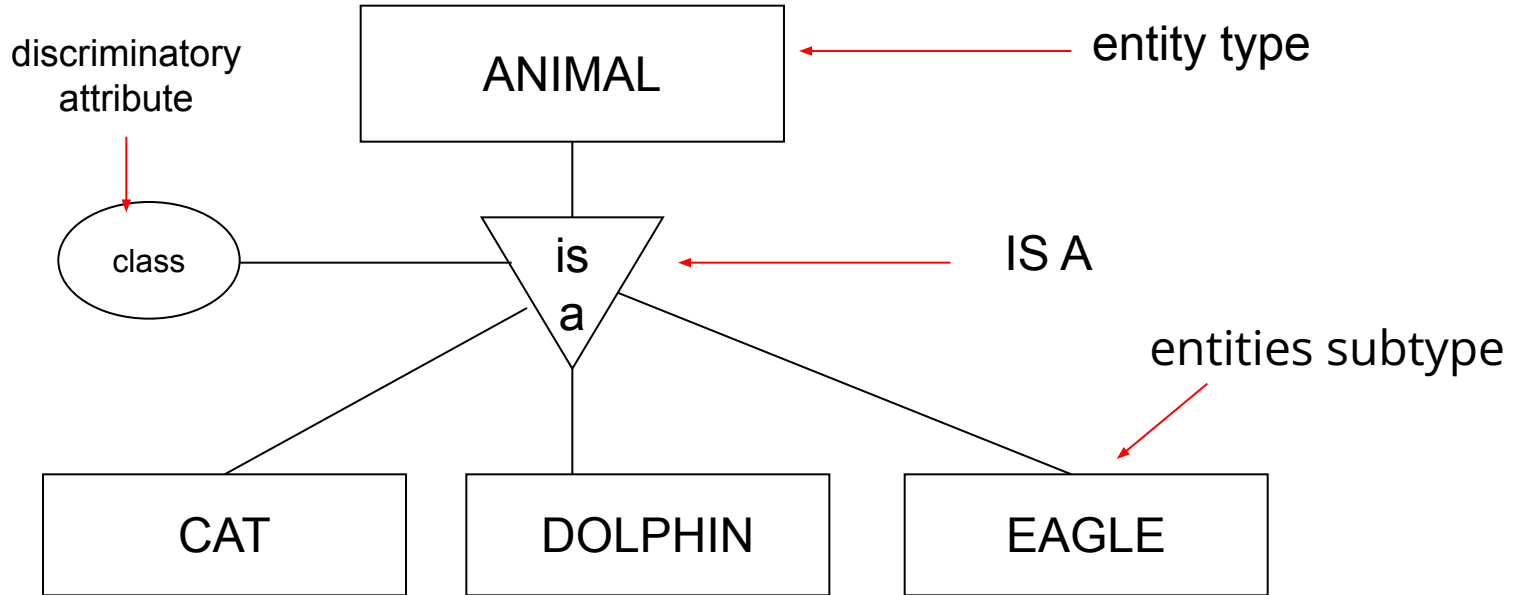
Scalable. It is possible to incorporate changes if new requirements need to be added.

Extended Entity-Relationship (EE-R) Model



Hierarchical relationships. IS A

They occur when one entity can be subdivided into other entities.

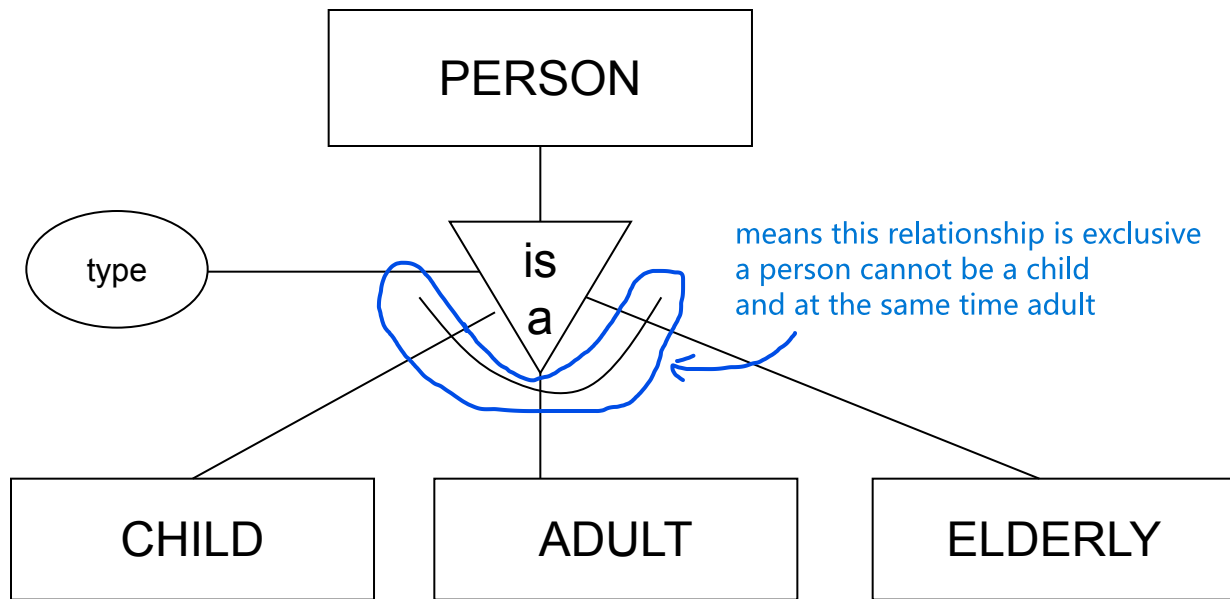


Hierarchical relationships. IS A

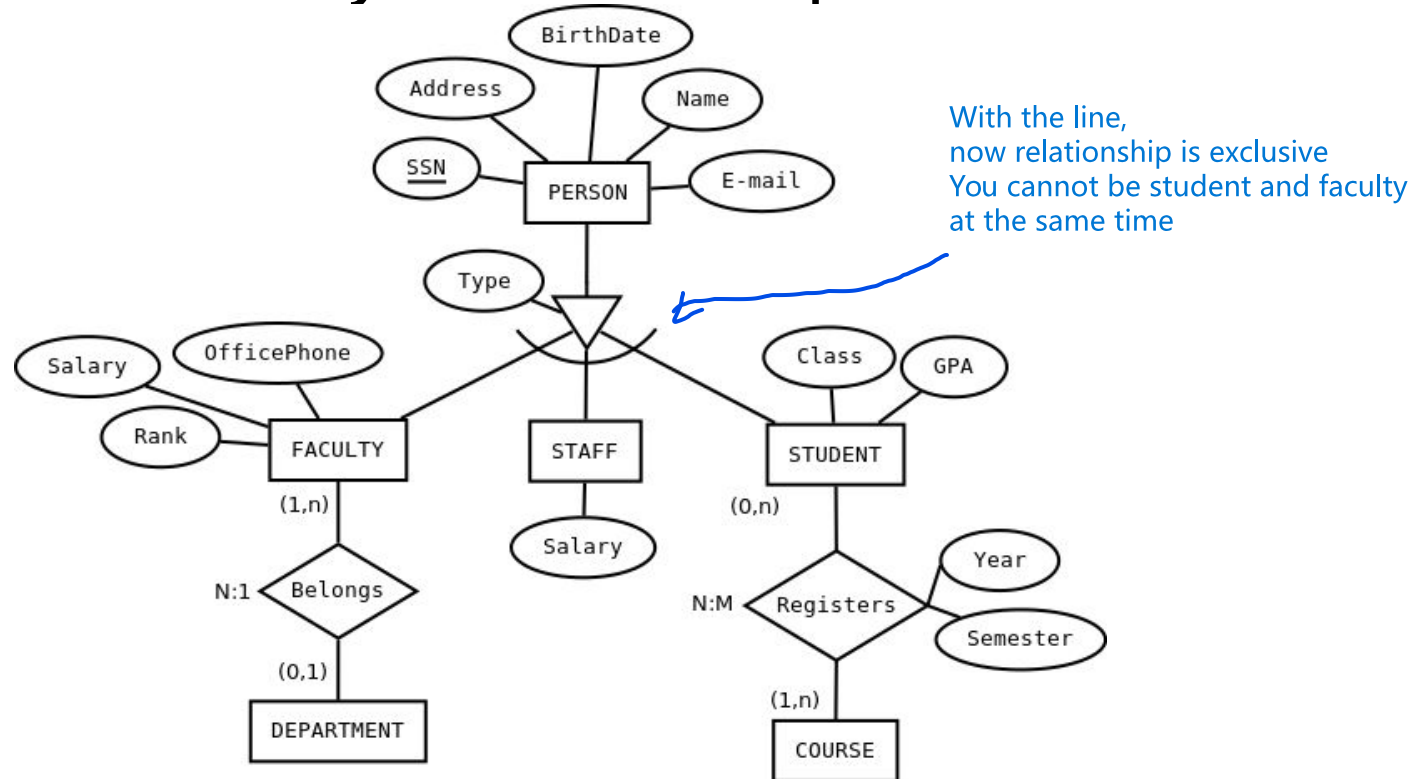
Types of hierarchical relationships.

(**Exclusive/overlapping**)

Exclusive. An instance of the supertype (superclass) can only appear in one subtype (subclass). It is represented by an arc crossing all the lines leading to the subtypes.



Extended Entity-Relationship (EE-R) Model

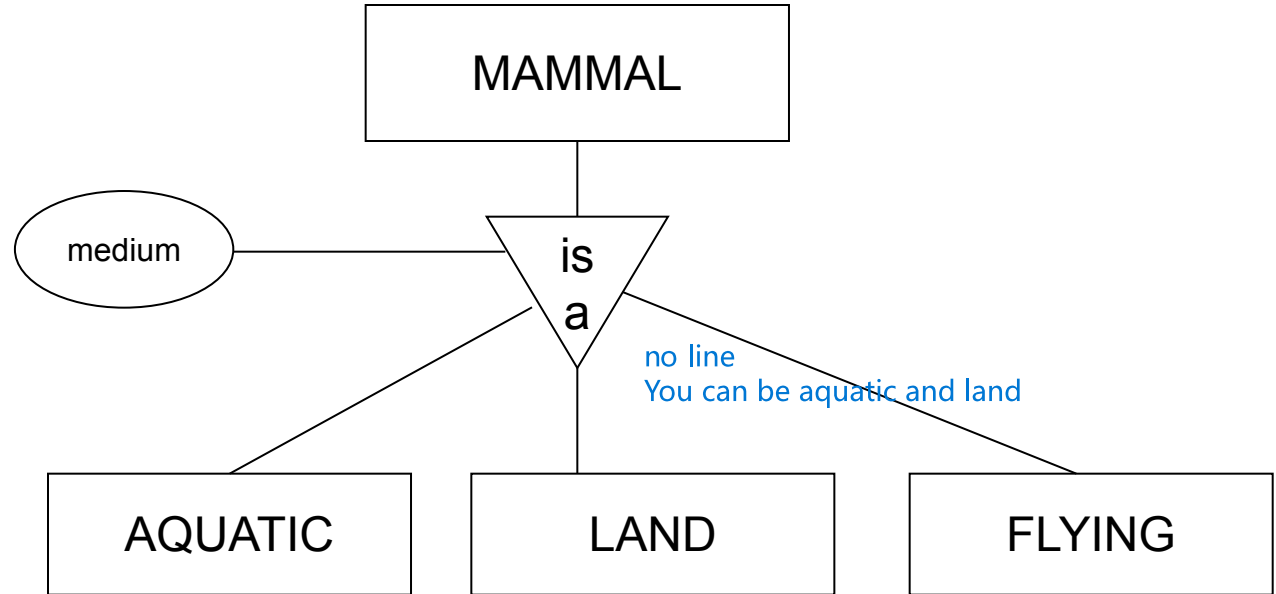


Hierarchical relationships. IS A

Types of hierarchical relationships.
(Exclusive/overlapping)

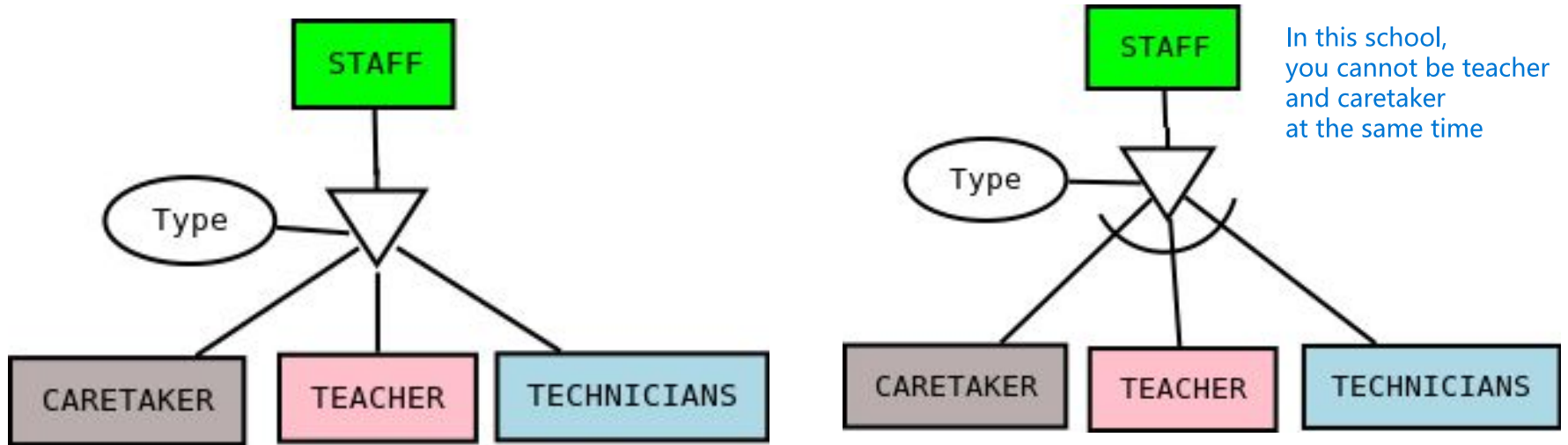
Overlapping. An instance of the supertype may appear in several entities of the subtype.

Overlapping = NOT exclusive



Hierarchical relationships. IS A

What's the difference?



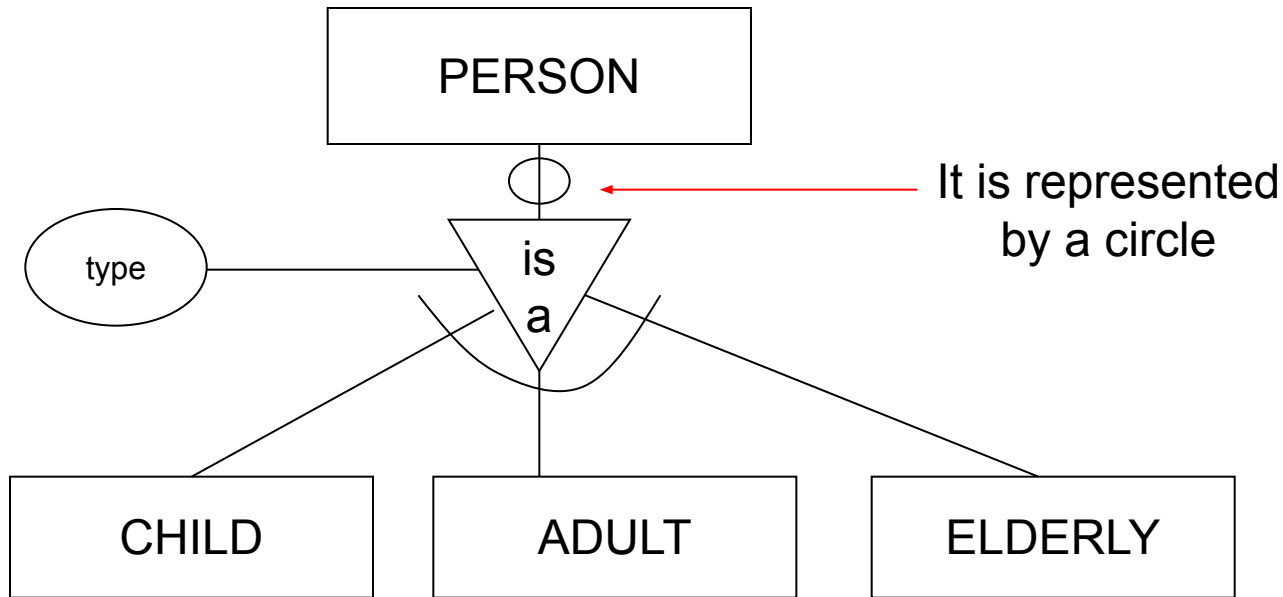
Hierarchical relationships. IS A

Types of hierarchical relationships
(**Total/Parcial**).

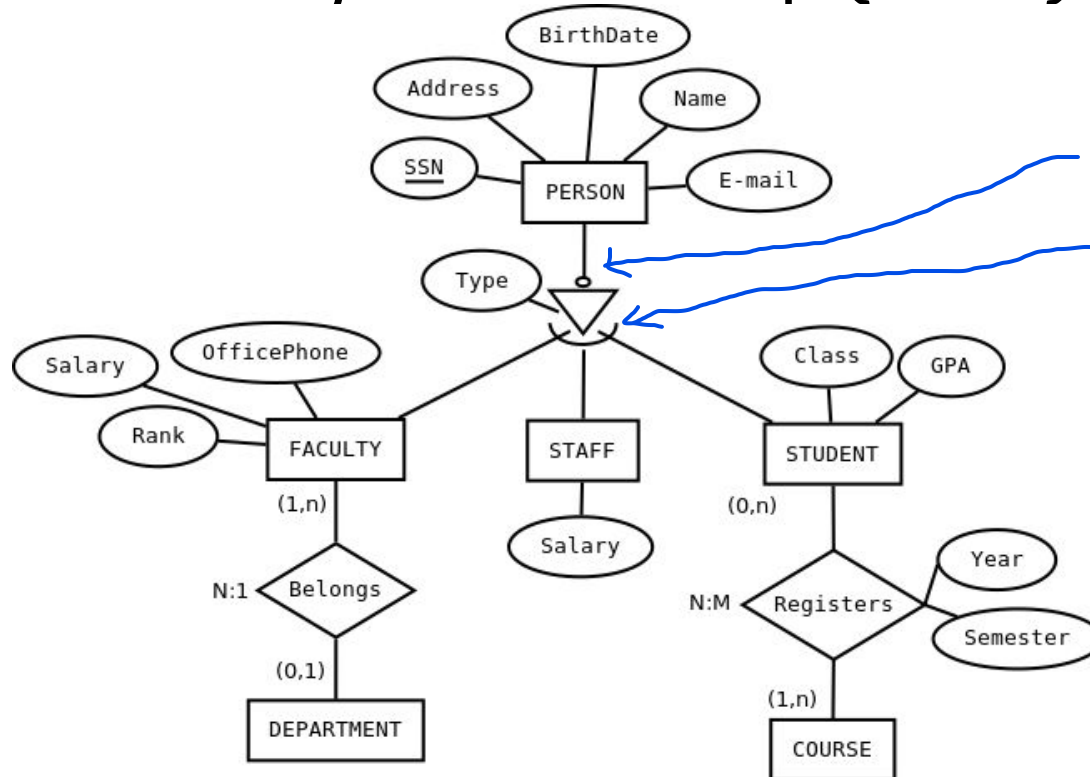
Total. All instances of the supertype must be represented in one of the subtypes.

A person must be one of child, adult, elderly
Cannot be baby.

States all the possibilities of Person



Extended Entity-Relationship (EE-R) Model

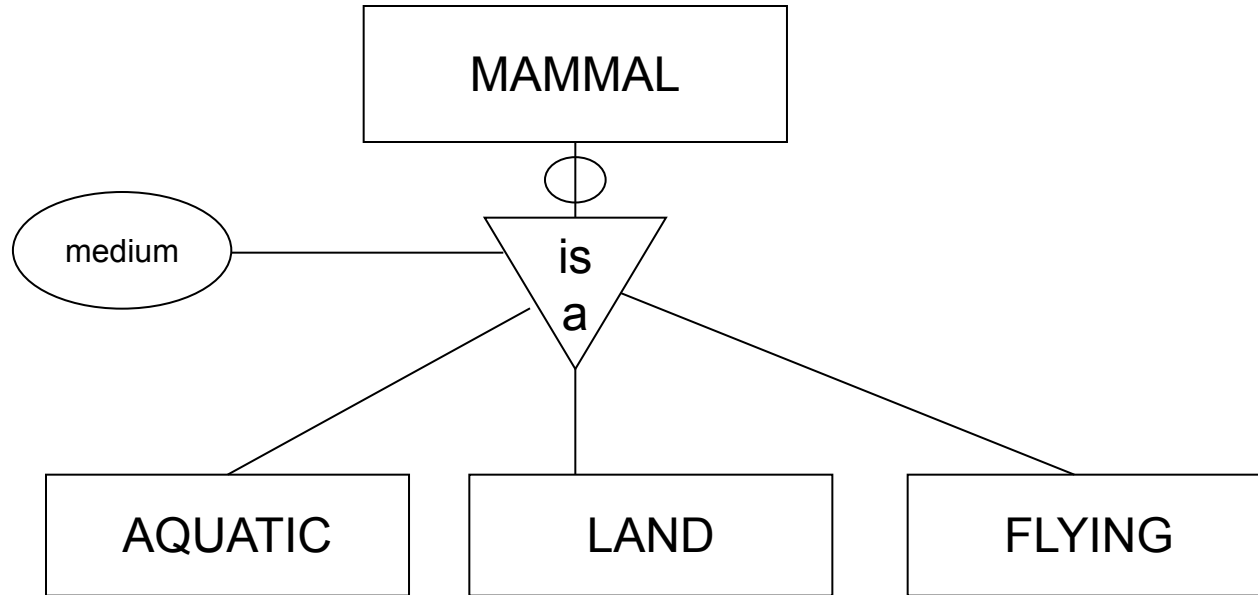


In this school,
you have to be one of
- faculty / staff / student
no more possibilities

And you can be just one rc

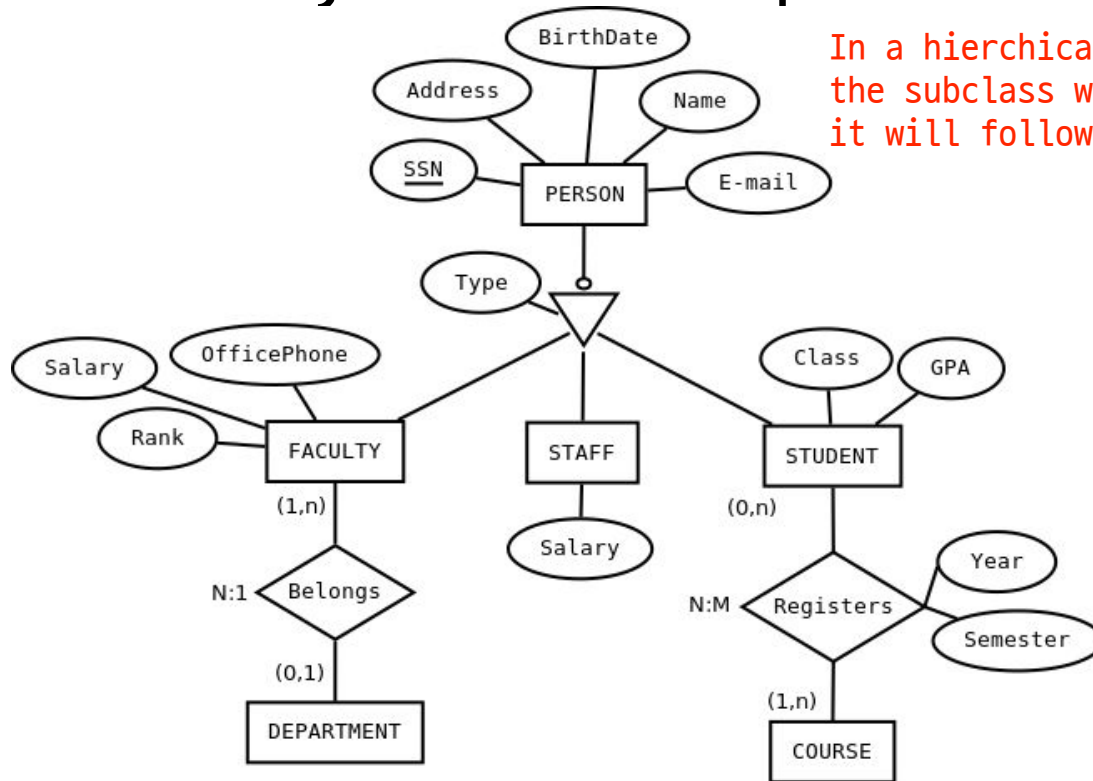
Hierarchical relationships. IS A

So ...



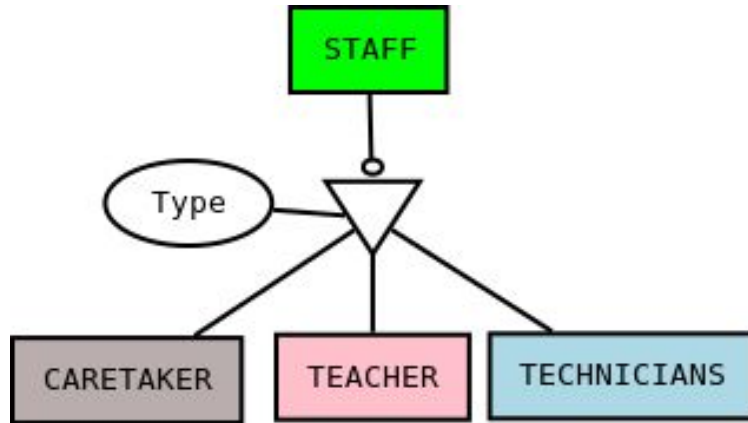
Extended Entity-Relationship (EE-R) Model

In a hierarchical relationship, the subclass will NOT have a primary key, it will follow the superclass primary key

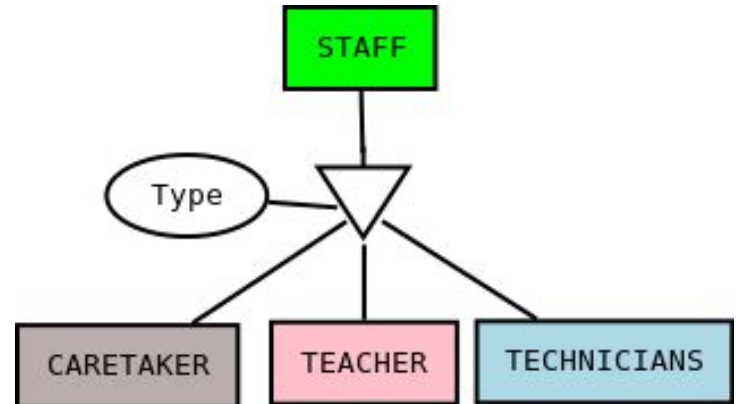


Hierarchical relationships. IS A

What's the difference?



You need to be one of three role
No other possibility of another j

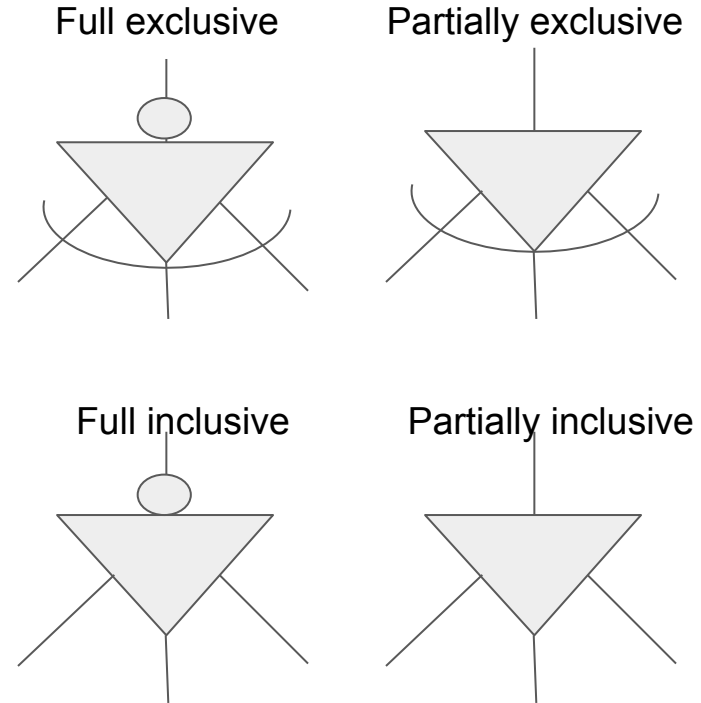


Hierarchical relationships

In summary, ...

a hierarchical relationship can be:

- Fully exclusive
- Partially exclusive
- Fully overlapping
- Partially overlapping



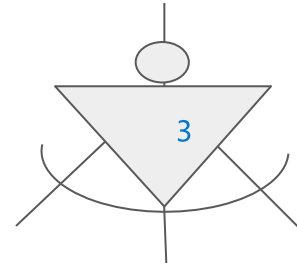
Hierarchical relationships

Examples:

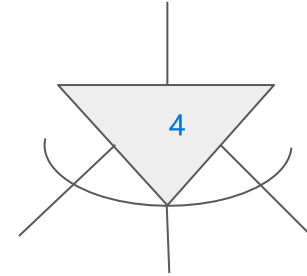
1. PERSON is_a CHILD, TEENAGER, ADULT, ANCIENT
2. PERSON is_a STUDENT, WORKER
3. DISEASE is VIRICAL, BACTERIAL
4. VEHICLE is TRUCK, PASSENGER



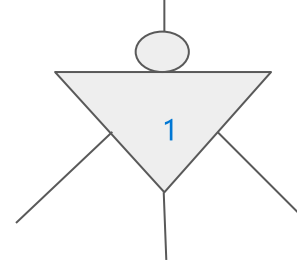
Full exclusive



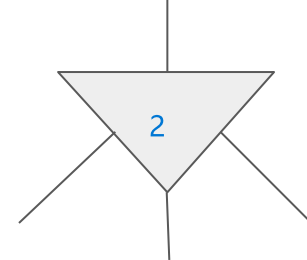
Partially exclusive



Full inclusive



Partially inclusive



Exercises

A car dealership sells **two types of vehicles**: passenger cars and trucks. The following information is stored for each vehicle: chassis number, datebuilt, price.

- For a truck, the following information is stored: number of axles and tonnage.
- For passenger cars, the following information is stored: door number.

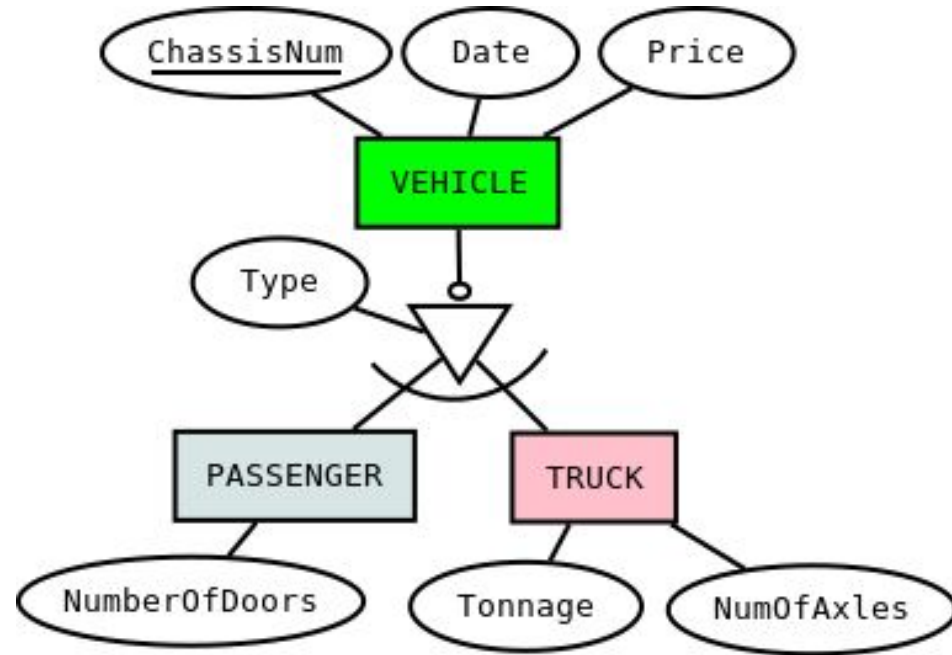
Represent with the E/R model this scenario.

Exercises

Any vehicle has the **attributes** chassis number, date of manufacture and price. Therefore, they are attributes of the superclass Vehicle.

A superclass Vehicle may only be either a PassengerCar subclass or a Truck subclass. It cannot be anything else. → **total**.

A vehicle cannot be both at the same time → **exclusive**.



Exercises (II)

- 1.- A company is divided into several departments identified by their names.
 1. The company's employees are known by their identity card number, their name and their home address. An employee is assigned to a certain department.
 2. If the employee is a driver, the driving licence number and the number of accidents he/she has had are known. If he/she is an administrative employee, the languages he/she speaks are known. If he/she is a technician, the specialities in which he/she can work are known.
 3. The company has a set of vehicles, of which the registration number and brand are known. These vehicles are driven by drivers, and any driver can be assigned to drive any vehicle on a given date (for the whole day, i.e. on any given day only one driver will drive it).
 4. The company runs a set of projects on which they work. These projects are identified by a project number and have an allocated budget. Several technicians can be assigned to a project. A technician may be assigned to several projects.

Exclusiveness (XOR)

- **Exclusiveness (XOR).** The existence of a relationship prevents the existence of another. Suppose an educational center where a teacher can teach or work on a research project, but not do both at the same time.

Heritage

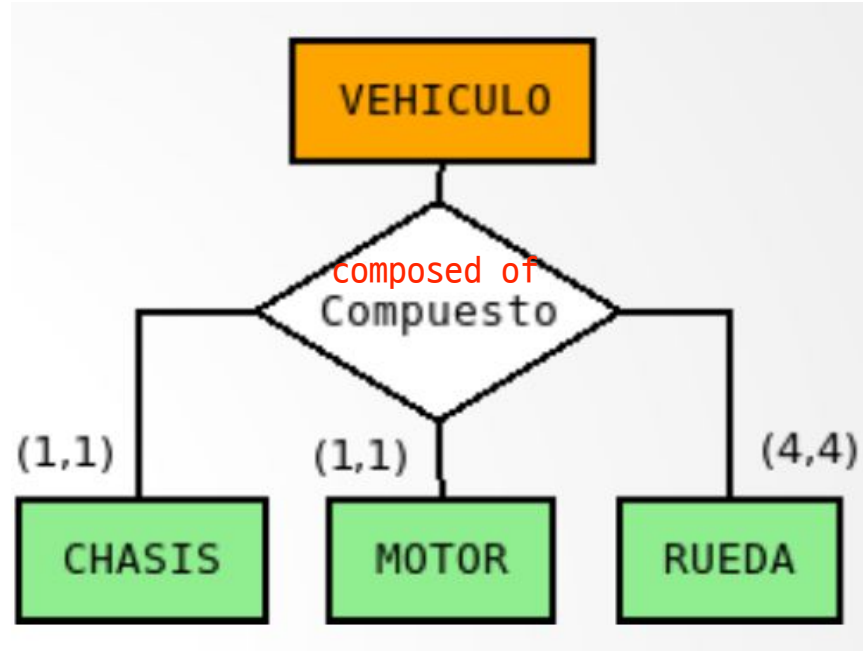
Represents a relationship between classes in which an entity is composed of one or more entities. There are two kinds:

Composite/Component. An entity is obtained by the union of several entities, which may be different objects and which play different roles.

Member/Collection. An entity is obtained by the union of several parts of the same type of the same type of entity and which play the same role in the of entity and playing the same role in the inheritance.

Heritage

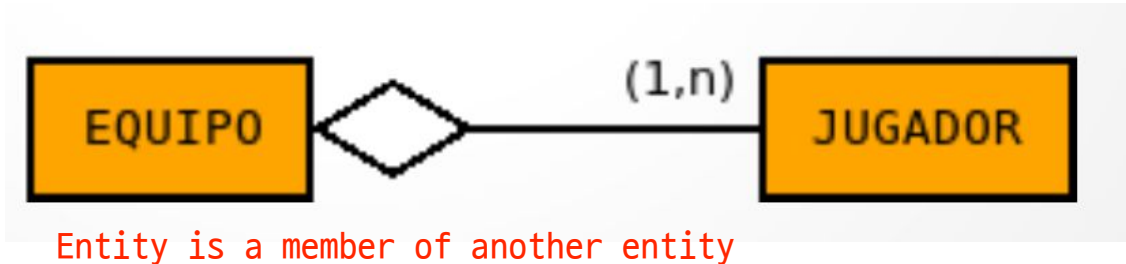
Composite/Component. An entity is **obtained** by the union of several entities, which may be different objects and which play different roles.



Heritage

Member/Collection. An entity is obtained by the union of several parts of the same type of entity and which play the same role in the of entity and playing the same role in the inheritance.

Therefore, this abstraction allows to represent a whole as a collection of members, all of the same entity type and all playing the same role.



NOT IN EXAM

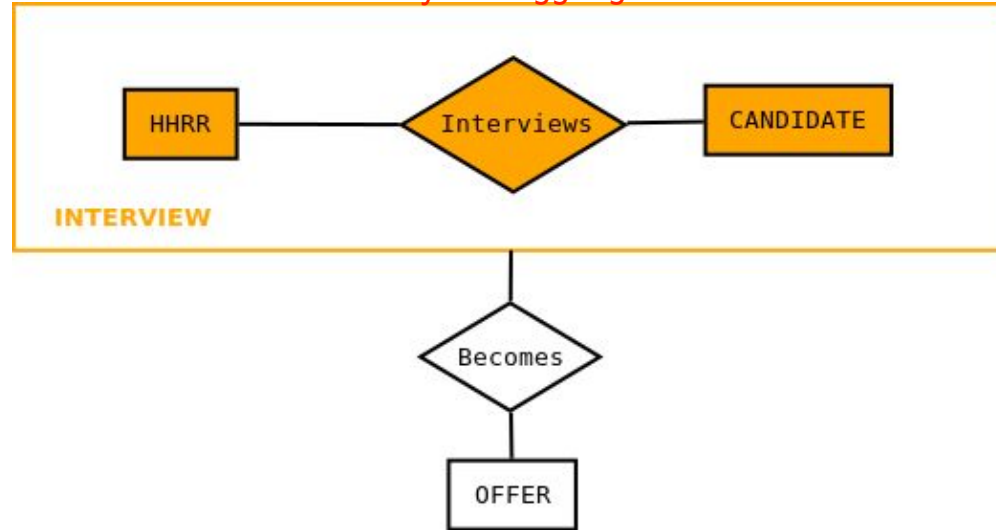
Aggregation

In the E/R model it is not possible to represent **relationships between relationships**.

Aggregation. Abstraction through which **relationships are treated as higher level entities**, being used to express relationships between relationships or between entities and relationships.

In aggregation, the **maximum and minimum cardinality** of the aggregated entity will always be **(1,1)**.

This becomes an entity in aggregation



What's the problem statement



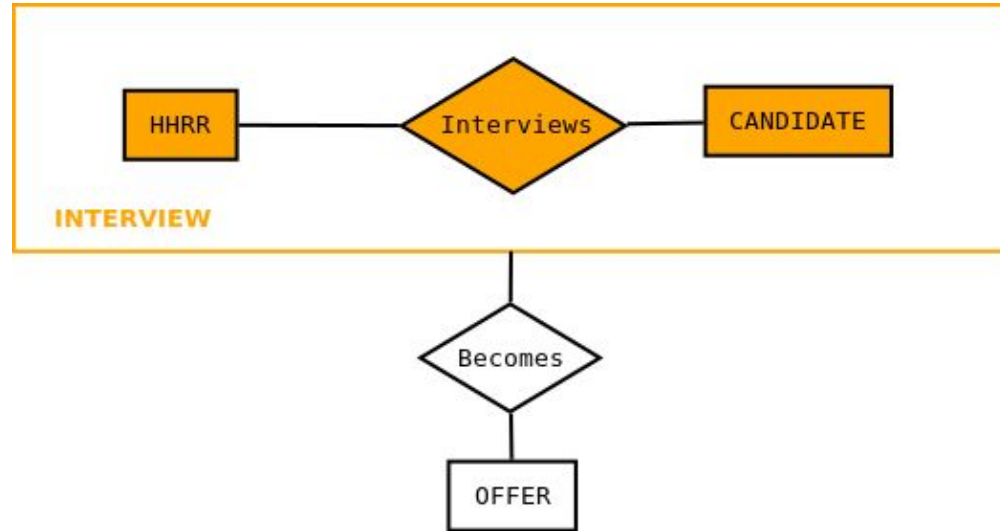
Aggregation

Problem statement

A recruitment company makes interviews with various applicants.

Some of these interviews result in a job offer.

The other interviews are stored in the system for future offers.



Exercises (II)

2.- It is desired to model the structure of a database containing information related to exercises for the whole **University**.

1. The exercises are identified by a code, with a date and can have several authors. These exercises can be practical exercises, exam-type exercises or simply suggested exercises (proposal).
2. Both practical exercises and exams are associated with subjects within particular degree programs. For example, the exercise code BDI274 is a practical exercise associated with the subject Database I, of the Systems Engineering degree, but it is not associated with the same subject of the Information Systems Technology degree program.
3. Regarding the practical exercises associated with a subject in a degree course, it is important to know on which sheet number the exercise is located, its order number within the sheet and an associated difficulty code. A practical exercise can appear in several subjects in different degree courses. All practical exercises are associated with a subject.
4. For exam-type exercises, it is of interest to know the date of the exam in which it was published, its maximum points and the number of the exercise in the exam. An exam-type exercise is associated with one and only one subject within a degree course.
5. The information of interest about the subjects is the name, and about the degree courses their identification.
6. It is known that within a degree course there are many subjects and that the same subject can be within several degree courses.
7. A subject in a degree course has a certain number of course hours per week. The subjects may have the same name, although they must have a unique code.
8. In the other hand, all exercises (of any kind) are related to topic with different degrees of difficulty.
9. Each topic is identified by a code, and it is of interest to know which topic precedes others.

Exercises (II)

Search for one aggregation

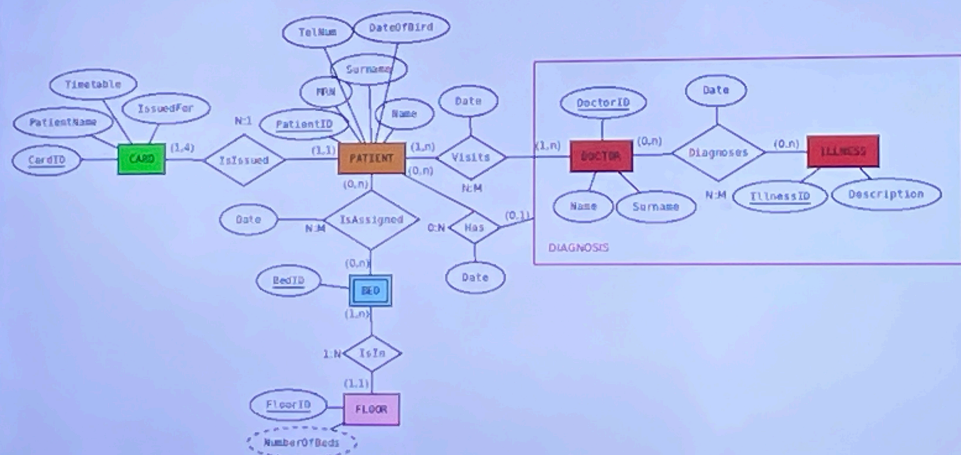


3.- A **hospital** centre wishes to computerise part of its patient management. Following the analysis carried out, the following requirements are established:

1. The patient data of interest to be stored are: patient number, medical record number, name, surname, address and telephone number of residence and date of birth.
2. A patient will be assigned to a specific bed on a specific hospital floor, and may be assigned to different beds and floors during the time of admission; the assigned date and bed number is important.
3. It should be kept in mind that beds are numbered consecutively for each floor, i.e. there will be bed number 12 on the third floor and also bed number 12 on the seventh floor. The hospital floors are identified by floor number, and number of beds available.
4. Up to a maximum of 4 visiting cards will be issued for each patient. These visiting cards shall be valid for visiting only one patient. The visiting card shall be defined by: visiting card number and the start and end time at which the patient may be visited; it must be known to whom it was issued and which patient may be visited.
5. A patient can be seen by different doctors, and the date and time of each visit is important. **Furthermore, a patient may have different diagnoses of illness, but only one in a specific date, so it is necessary to know the date of diagnosis.**
6. On the other hand, one doctor can treat different types of illness and vice versa. The data of interest of the doctors will be: doctor's code, first name and surname.
7. The data of interest for illness will be: illness code and description.
8. Once a patient has been discharged (allowed to leave), all the information relating to the patient is transferred to a historical archive.

POR FAVOR,
CUIDEMOS LAS INSTALACIONES
CON UNA AYUDA
PARA NUESTRA FORMACIÓN

Exercises (II)



References

- <https://ikastaroak.birt.eus/edu/argitalpen/backupa/20200331/1920k/es/DAMDAW/BD/BD02/es DAMDAW BD02 Contenidos/website index.html>
- <https://ikastaroak.birt.eus/edu/argitalpen/backupa/20200331/1920k/es/DAMDAW/BD/BD02/es DAMDAW BD02 Contenidos/website 41 tipos de a tributos.html>
- http://dia-installer.de/howto/create_shape/index.html.en (use F9 to add shape)